



Máster en
Inteligencia
Artificial
UMU

Deep Learning para el procesamiento de Lenguaje Natural

Tema 1. Introducción al Deep Learning.
Redes Neuronales

JUAN A. BOTÍA
CIENCIA DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL
DEPT. INGENIERÍA DE LA INFORMACIÓN Y LAS COMUNICACIONES

Universidad de Murcia

Máster en Inteligencia Artificial

2026/2027



Índice de contenidos



Máster en
Inteligencia
Artificial
UMU

- Los problemas a resolver con esta herramienta
- El perceptron
- El perceptron multi-capas
- Aplicación en problemas relacionados con texto
- Optimización



Empecemos por el principio: ¿cuál es el problema que queremos resolver?

Veámos una serie de preguntas ejemplo que reflejan la tarea que afrontamos.

¿Es éste un email spam?



Máster en
Inteligencia
Artificial
UMU

Good morning Dan,

Please familiarize yourself with the attached file.
Reply here if you have any questions.

Thank you.

John and Mike,

Appreciate your flexibility this week, as the team navigates the sensitivities surrounding some of the project work taking place at the sites. Please tentatively plan for mobilization on 05/16/2022, in order to begin the final stages of the upgrade.

I will follow-up tomorrow with a confirmation if all indications are we will be given the “all-clear” before EOB Wednesday/SOB Thursday.

Appreciate your support.

Regards,



Judy Sewell
Project Manager



¿Son positivas o negativas estas reviews sobre películas?



- unbelievably disappointing



- Full of zany characters and richly applied satire, and some great plot twists



- this is the greatest screwball comedy ever filmed



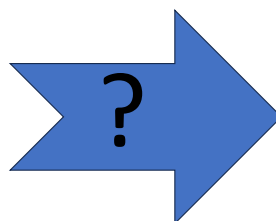
- It was pathetic. The worst part about it was the boxing scenes.

¿En qué disciplina se encuadra este paper?



Máster en
Inteligencia
Artificial
UMU

MEDLINE Article



MeSH Subject Category Hierarchy

- Antagonists and Inhibitors
- Blood Supply
- Chemistry
- Drug Therapy
- Embryology
- Epidemiology
- ...



La tarea genérica se denomina *clasificación de textos*, para...

- Asignación de categorías, tópicos, géneros...
- Detección de spam
- Identificación de autores
- Identificación de lenguaje
- Análisis de sentimientos
- ...



Text Classification: definición formal en forma funcional

- *Input:*

- un documento d
- *un conjunto de clases fijo* $C = \{c_1, c_2, \dots, c_J\}$

- *Output:* una de las clases $c \in C$



Text Classification: medir del rendimiento de un modelo en esta tarea

- *Supongamos una clase P(ositiva) y una clase N(egative)*
- *Posibles resultados según la celda*
 - *TP (true positive),*
 - *TN (true negative),*
 - *FP (false positive),*
 - *FN (false negative)*

Un ejemplo de matriz de confusión (contraste entre predicciones del modelo y respuestas verdaderas): 50 + 10 ejemplos positivos, 35 + 5 negativos

	Predicted Positive	Predicted Negative
Actual Positive	TP = 50 (True Positive)	FN = 10 (False Negative)
Actual Negative	FP = 5 (False Positive)	TN = 35 (True Negative)



Método básico (baseline) : reglas definidas a mano

- Basadas en la aparición de palabras u otras propiedades, como
 - spam: black-list-address OR (“euros” AND “has sido seleccionado”)
- La precision (tasa de aciertos) puede ser alta
 - Cuando el dominio es muy específico
 - Las reglas son definidas por expertos y se actualizan con frecuencia

Pero...

- La construcción de esas reglas y su mantenimiento es muy costoso
- Un enfoque basado en lo literal y lo específico suele estar destinado al fracaso:
 - Cuando detectan algo no suelen equivocarse (“high-precision”)
 - Muchas cosas escapan a su detección (“low-recall”)



Método Avanzado : Machine Learning Supervisado

- *Input:*

- Un tipo de documento d
- *Un conjunto de clases fijo* $C = \{c_1, c_2, \dots, c_j\}$
- Un conjunto M de entrenamiento de m documentos etiquetados a mano $(d_1, c_1), \dots, (d_m, c_m)$

- *Output:*

- Un clasificador aprendido a partir de M , $f: d \rightarrow c$



Método Avanzado : Machine Learning Supervisado

- Existen muchos tipos de técnicas (asignatura ML)
 - Bayes naive,
 - Regresión logística,
 - K-vecinos más cercanos y
 - Neural networks...

También Podemos usar LLMs, lo veremos más adelante!

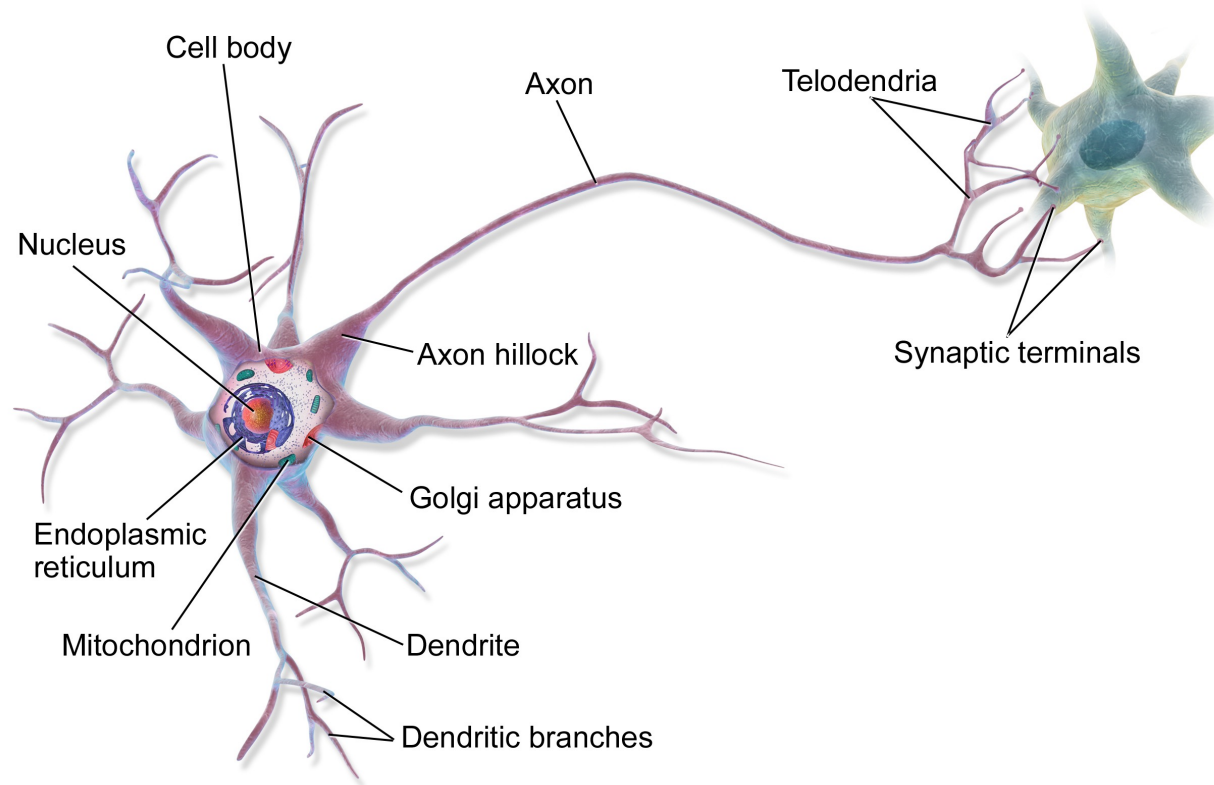
- Fine-tuned como clasificadores
- Usados para producir como salida una clasificación



Ahora que hemos visto el problema que queremos resolver... ¿nos centramos en las redes neuronales?



Esto es una neurona en tu cerebro

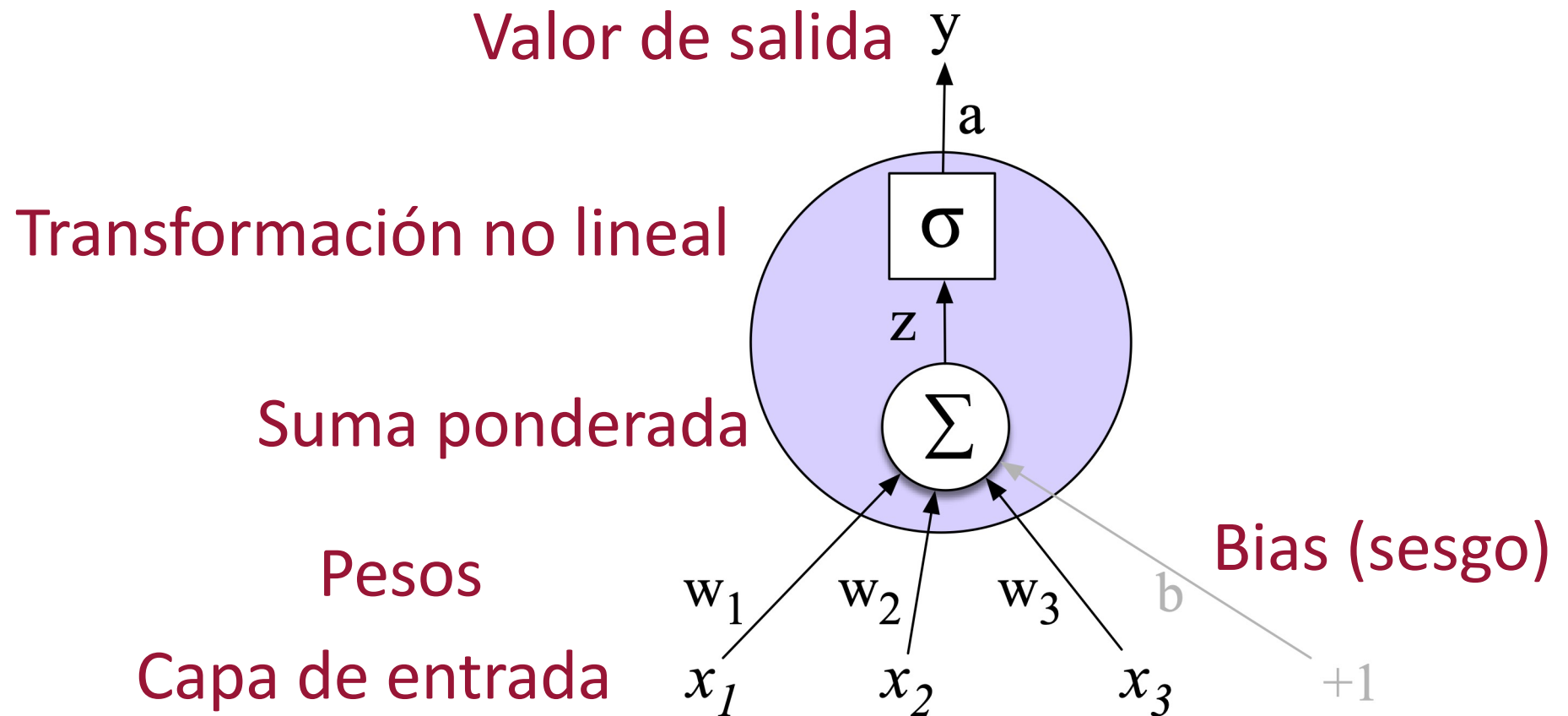


Esto es una neurona artificial

No es tu cerebro!



Máster en
Inteligencia
Artificial
UMU





Unidad neuronal

- Realiza sumas ponderadas, más un sesgo (término b , de *bias*)

$$z = b + \sum_i w_i x_i$$

$$z = w \cdot x + b$$

- Pero no nos basta con usar z , sino que además le aplicamos una función adicional no lineal *

$$y = a = f(z)$$

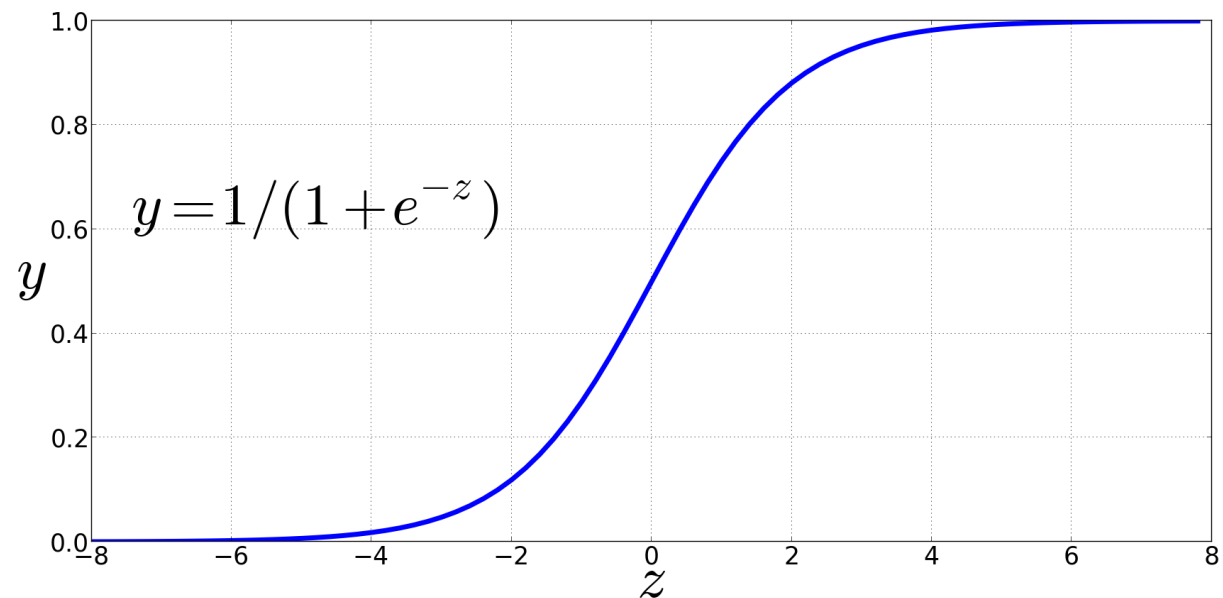
* Una función lineal era aquella cuya variable dependiente tomaba grado cero o uno, representando un punto o una línea recta, respectivamente

Funciones de activación no lineales: ejemplo



Sigmoide

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

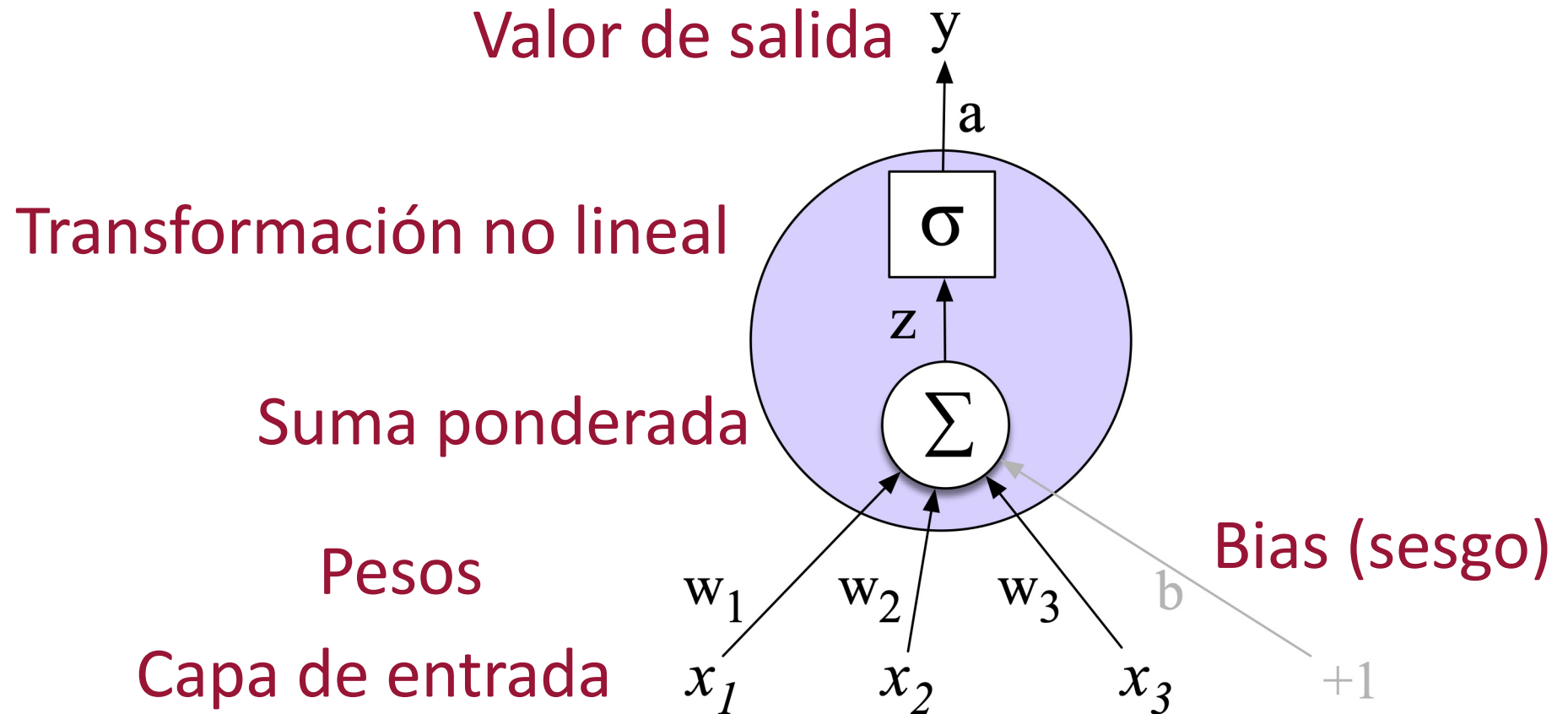


Y esta es la función final computada por la unidad neuronal



$$y = \sigma(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

Recordemos la neurona artificial!!

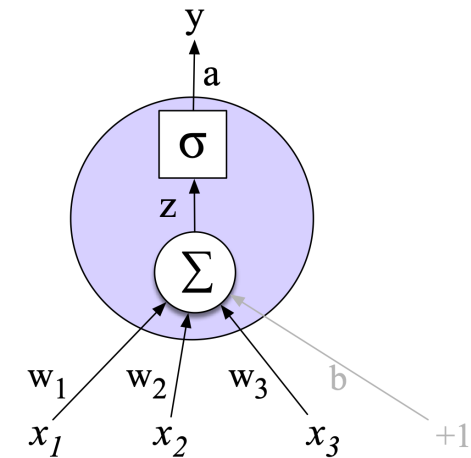


Y ahora un ejemplo



- *Supongamos que una neurona tiene la configuración actual siguiente:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- Y toma la entrada x que sigue:
- $x = [0.5, 0.6, 0.1]$

$$y = \sigma(w \cdot x + b) =$$

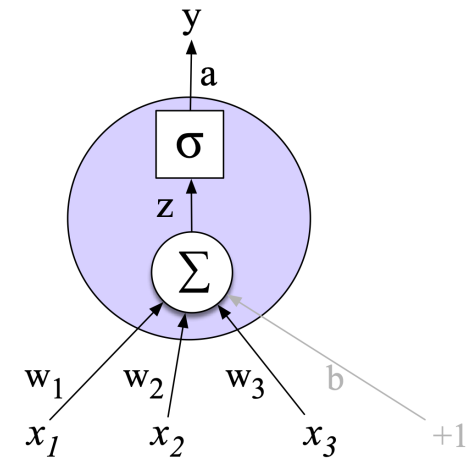




Y ahora un ejemplo

- *Supongamos que una neurona tiene la configuración actual siguiente:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- Y toma la entrada x que sigue:
- $x = [0.5, 0.6, 0.1]$

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

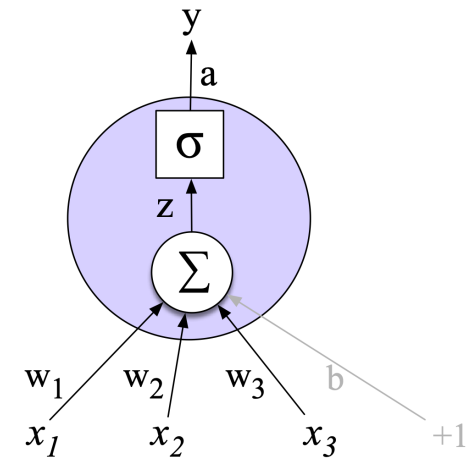




Y ahora un ejemplo

- *Supongamos que una neurona tiene la configuración actual siguiente:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- Y toma la entrada x que sigue:
- $x = [0.5, 0.6, 0.1]$

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} = \frac{1}{1 + e^{-(.5 \cdot .2 + .6 \cdot .3 + .1 \cdot .9 + .5)}} =$$

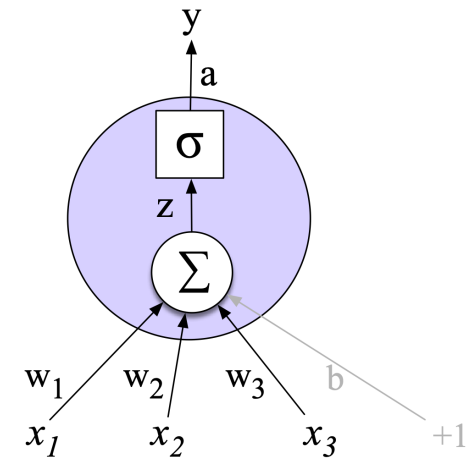




Y ahora un ejemplo

- *Supongamos que una neurona tiene la configuración actual siguiente:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- Y toma la entrada x que sigue:
- $x = [0.5, 0.6, 0.1]$

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} = \frac{1}{1 + e^{-(.5 * .2 + .6 * .3 + .1 * .9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

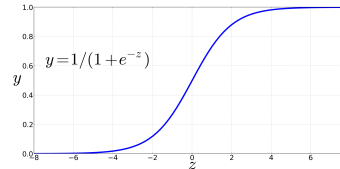


Otras funciones no lineales usadas

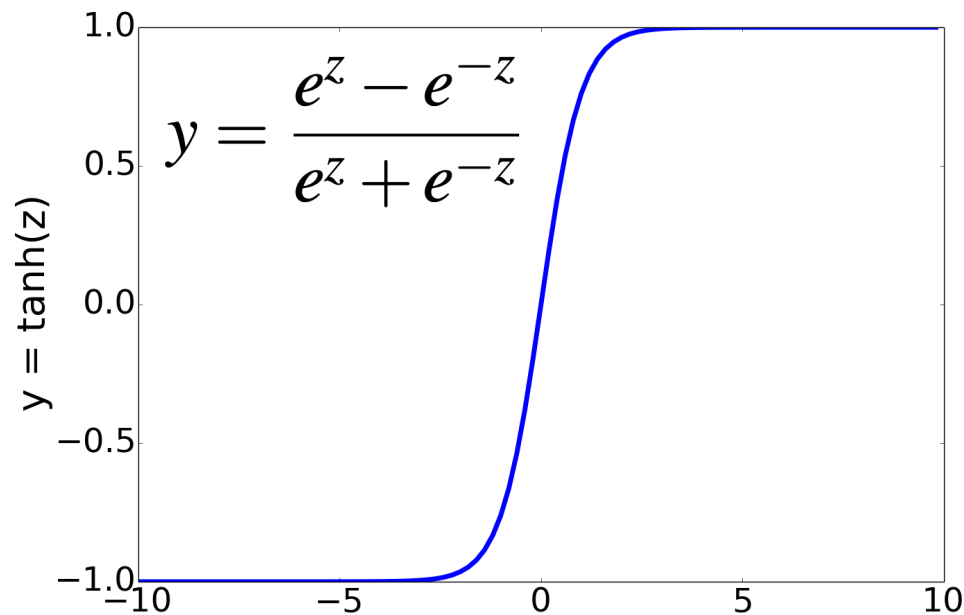


Máster en
Inteligencia
Artificial
UMU

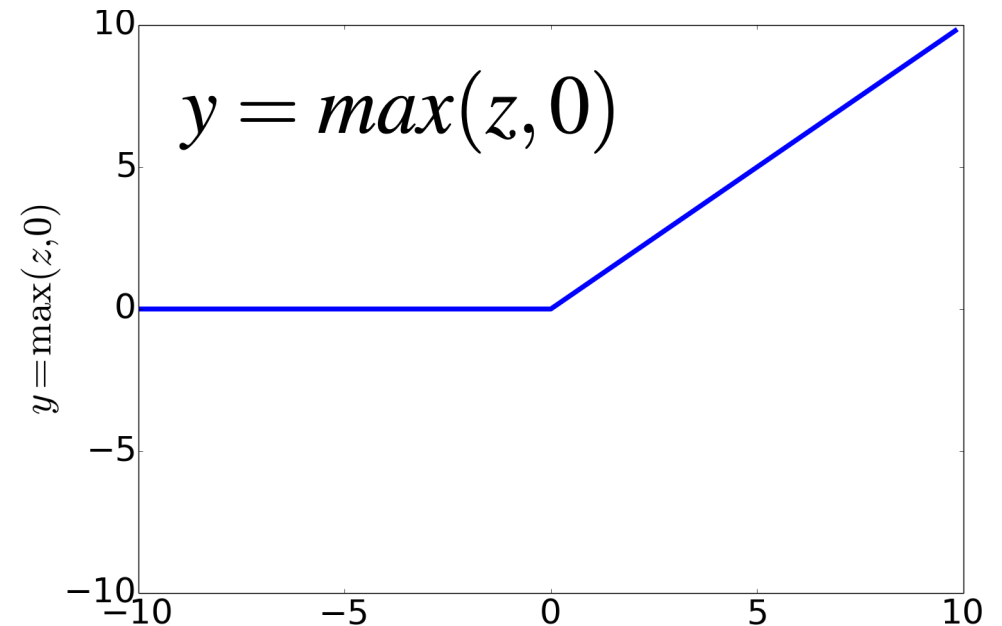
sigmoidal



La más común ahora



tanh



ReLU

Rectified Linear Unit





Ya sabemos cómo funciona una unidad básica.
Ahora ¿para qué sirven? ¿cómo las podemos usar?



El problema del OR exclusivo (XOR)

Minsky and Papert (1969)

- Lo primero para lo que se pensó usarlas era como puertas lógicas y así manipular conocimiento básico
 - Limitamos la entrada a 1's (true) y 0's (false)
 - Primera dificultad: necesitamos generar 1's y 0's a la salida

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

Primera dificultad: necesitamos
generar 1's y 0's a la salida



Adaptación de la unidad neuronal para generar salidas binarias

Simplificamos la definición

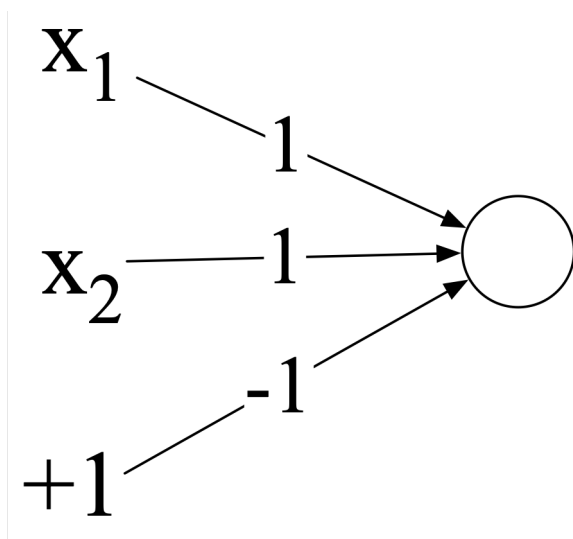
- Limitamos la salida: binary output (0 or 1)
- Eliminamos la función de activación

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

Operaciones AND y OR

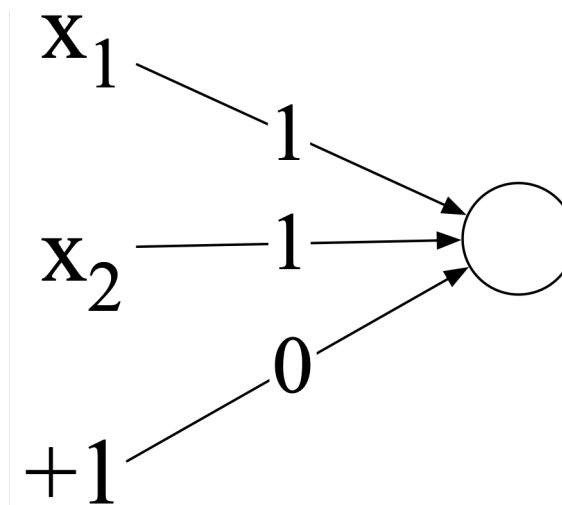


$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$



AND

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



OR

OR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1



Y la negación?

- $x=0$: $-1 \times 0 + 0.5 = 0.5 \geq 0$ entonces $y = 1$
- $x=1$: $-1 \times 1 + 0.5 = -0.5 < 0$ entonces $y = 0$

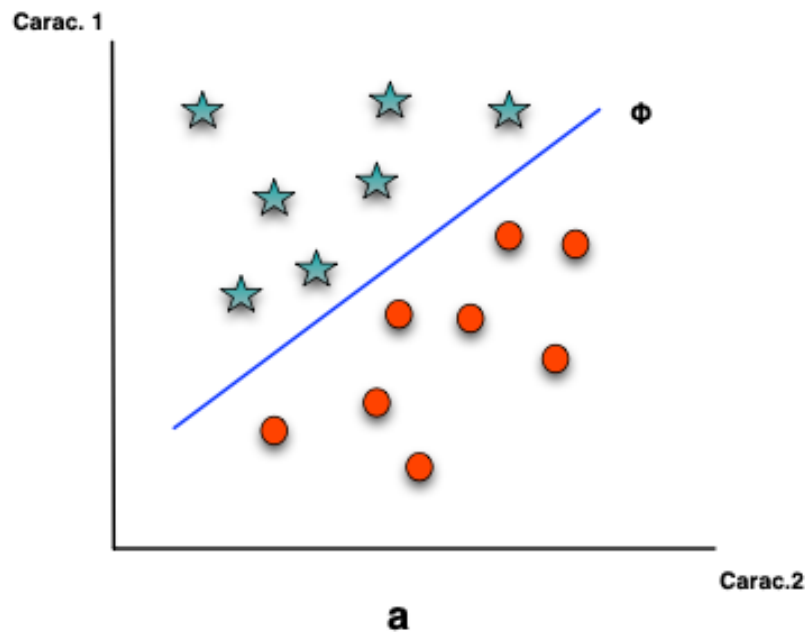


Es imposible reproducir un XOR con un perceptrón

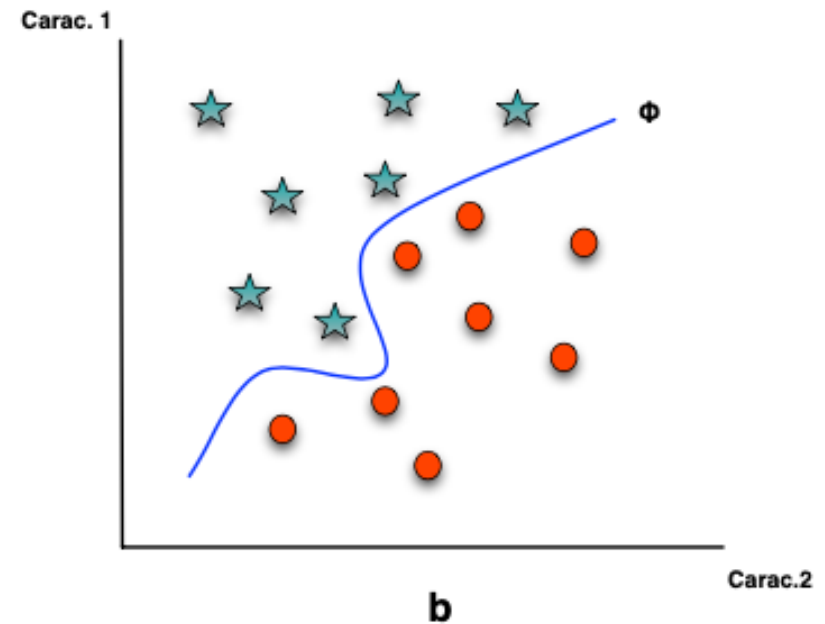
- Inténtalo tú mismo

XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

¿Por qué? Los perceptrones son máquinas lineales



Problema de clasificación linealmente separable



Problema de clasificación no separable linealmente



Máquinas lineales

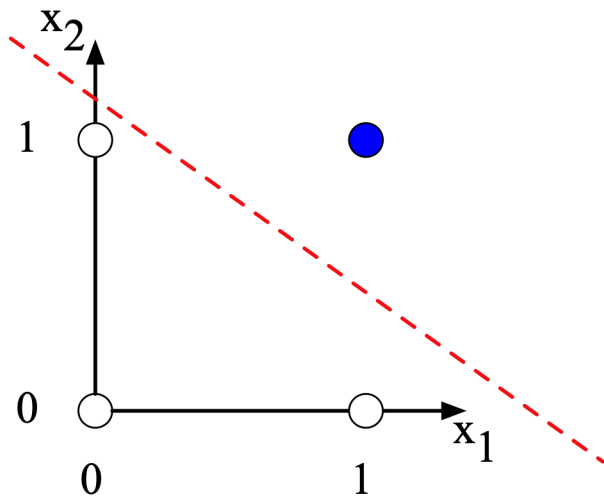
- La ecuación interna básica del perceptrón para variables x_1 y x_2 , es

$$w_1x_1 + w_2x_2 + b = 0$$

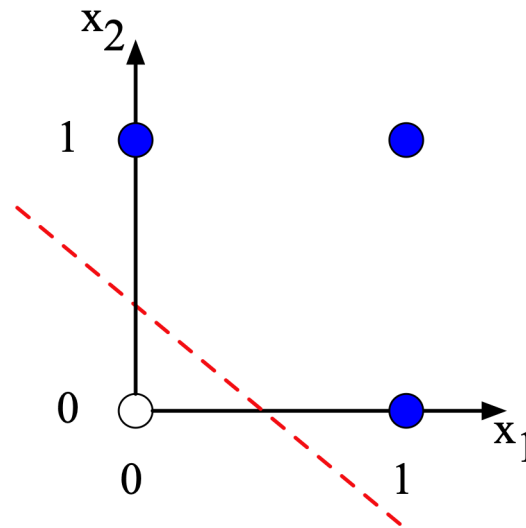
- Si queremos usarla para representar la línea, usamos $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$
- Como hemos visto, la línea es una franja de decisión (i.e. **decision boundary**)
 - 0 si la entrada cae a un lado de la línea
 - 1 si cae al otro lado



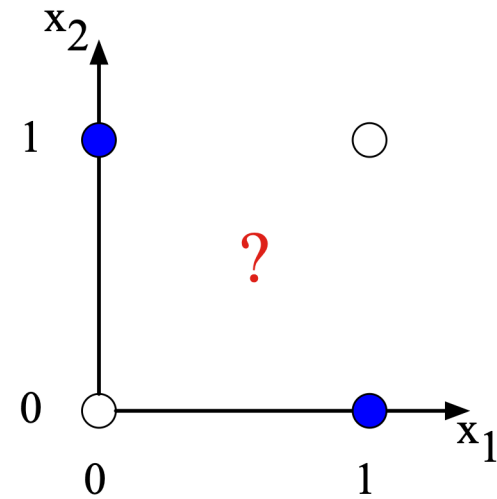
Decision boundaries



a) x_1 AND x_2



b) x_1 OR x_2



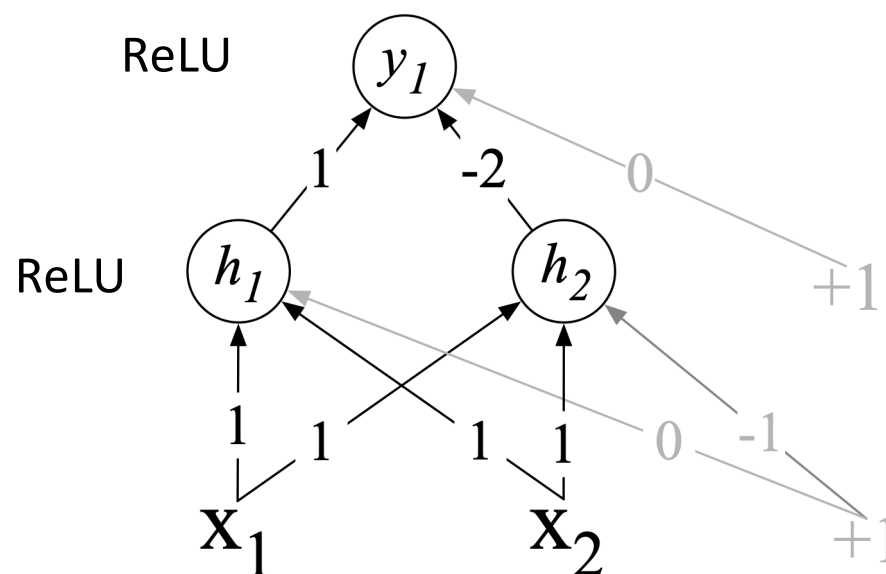
c) x_1 XOR x_2

XOR no es una función linealmente separable!

Solución al XOR, añadir más perceptrones



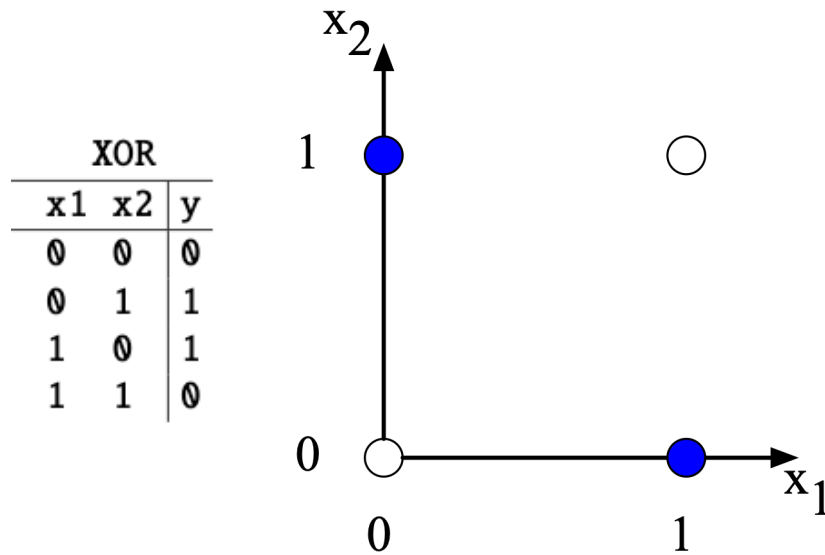
XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



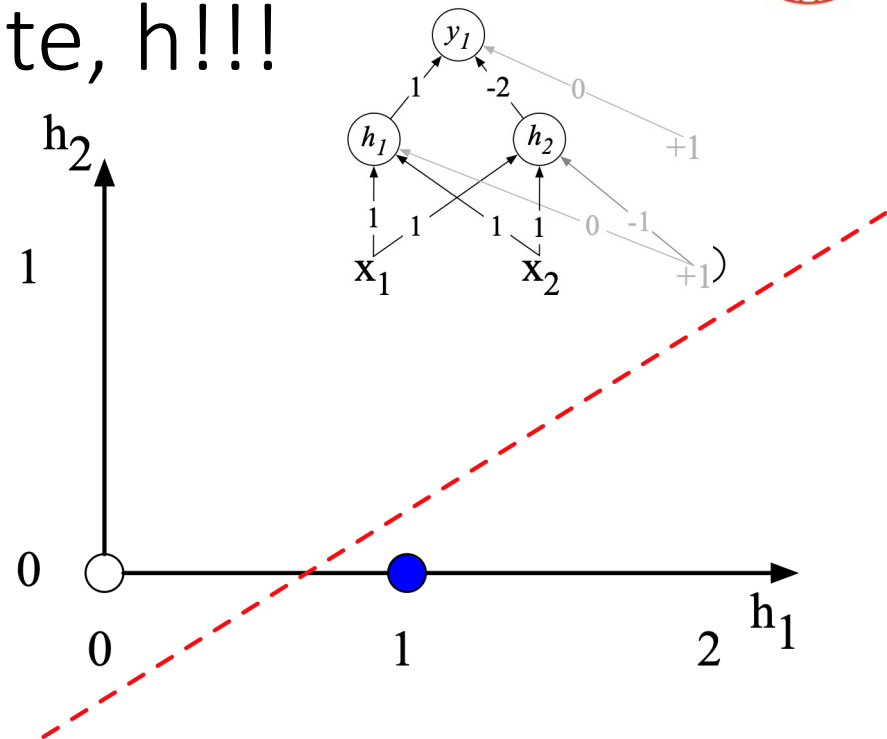


Interesante!!!

La representación latente, h !!!



a) The original x space



b) The new (linearly separable) h space

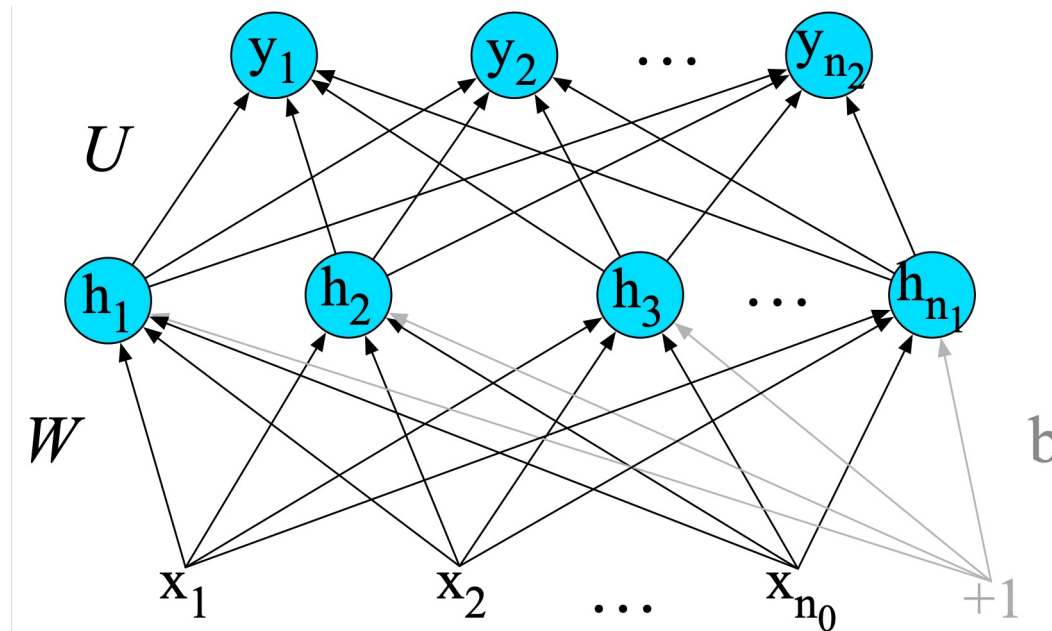
(Primer ejemplo de aprendizaje: capas ocultas aprenden formas útiles y nuevas de representar los datos)



Redes neuronales feedforward

Feedforward Neural Networks

- También denominadas perceptrones multi-capas (o **MLPs**) por razones históricas



Un perceptron con función de activación sigmoideal puede verse como una red de 1 capa

La capa de entrada no se cuenta como tal



Output layer
(σ node)

$$y = \sigma(w \cdot x + b)$$

(y es un escalar)

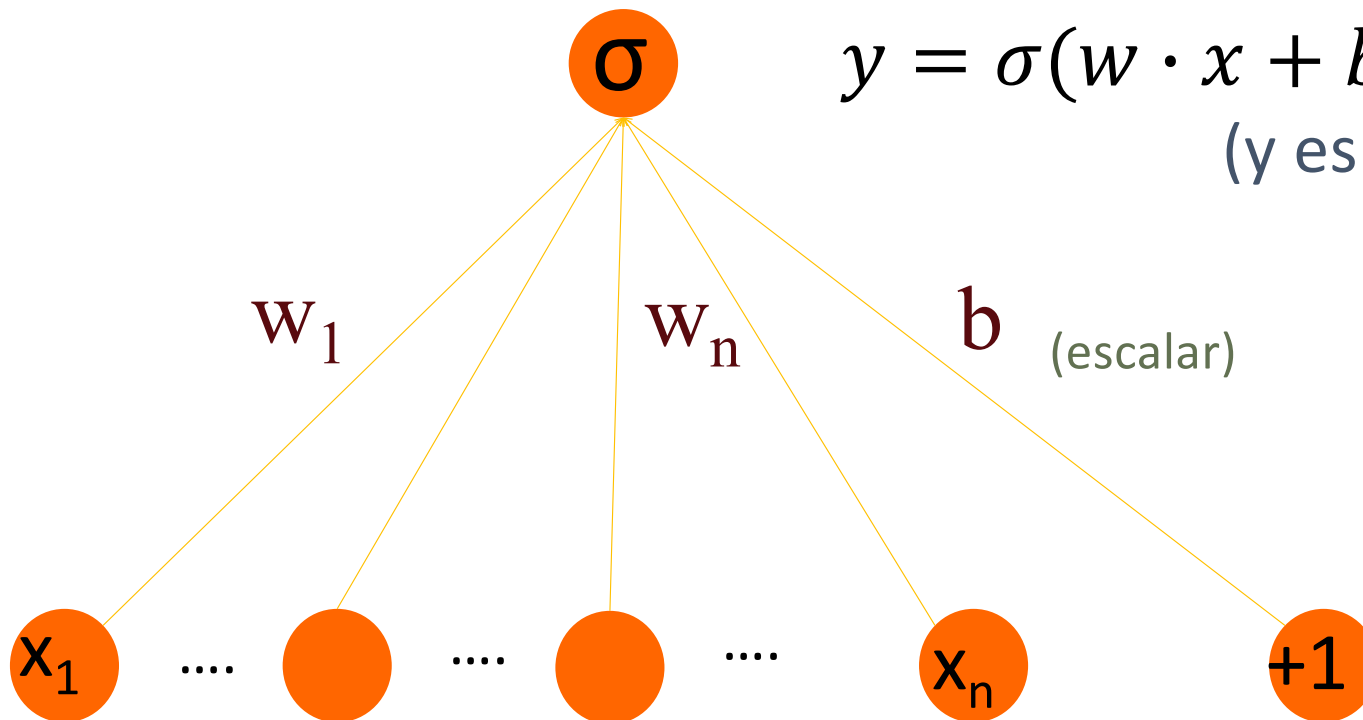
W
(vector)

w_1

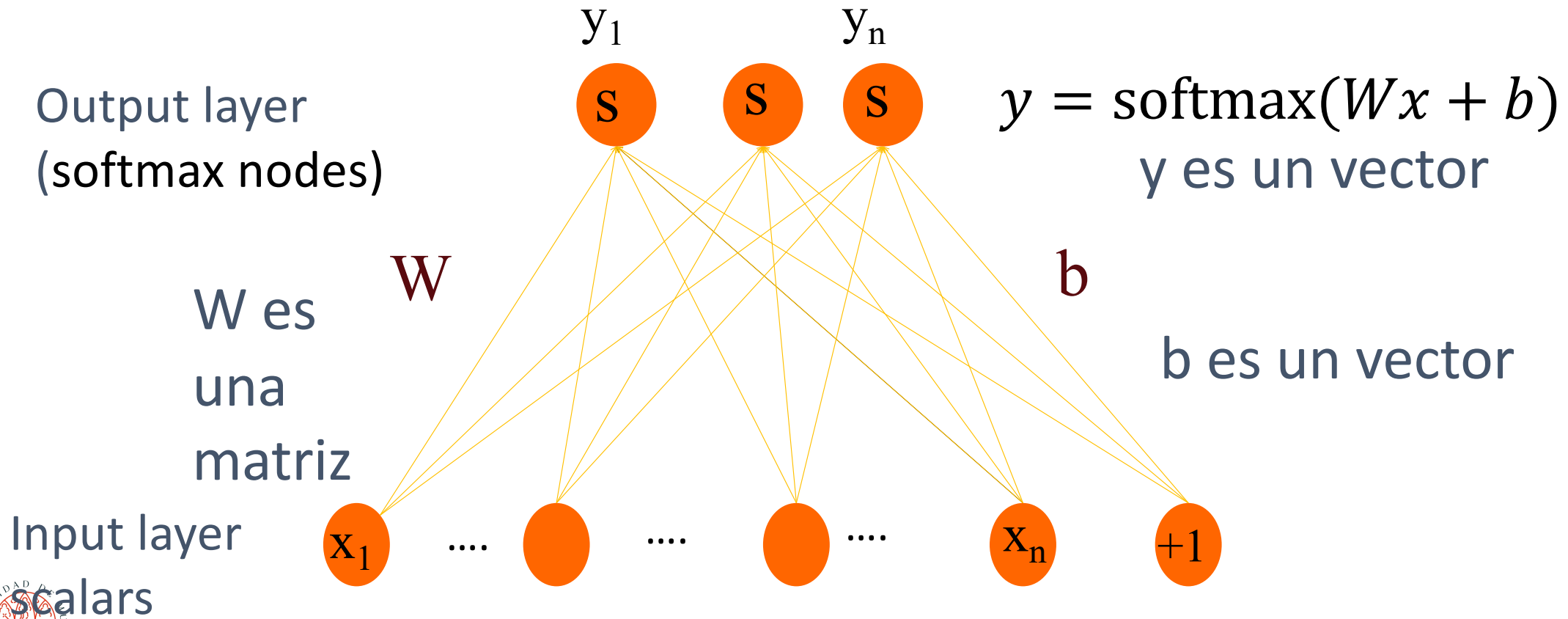
w_n

b
(escalar)

Input layer
vector x



Un perceptron con función de activación sigmoideal puede verse como una red de 1 capa **Fully connected single layer network**



El softmax es una generalización de la sigmoide



- Para un vector z de dimensiones k , el softmax es:

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^k \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^k \exp(z_i)}, \dots, \frac{\exp(z_k)}{\sum_{i=1}^k \exp(z_i)} \right]$$

- Example:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)} \quad 1 \leq i \leq k$$

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z) = [0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$

Red de dos capas con salida escalar

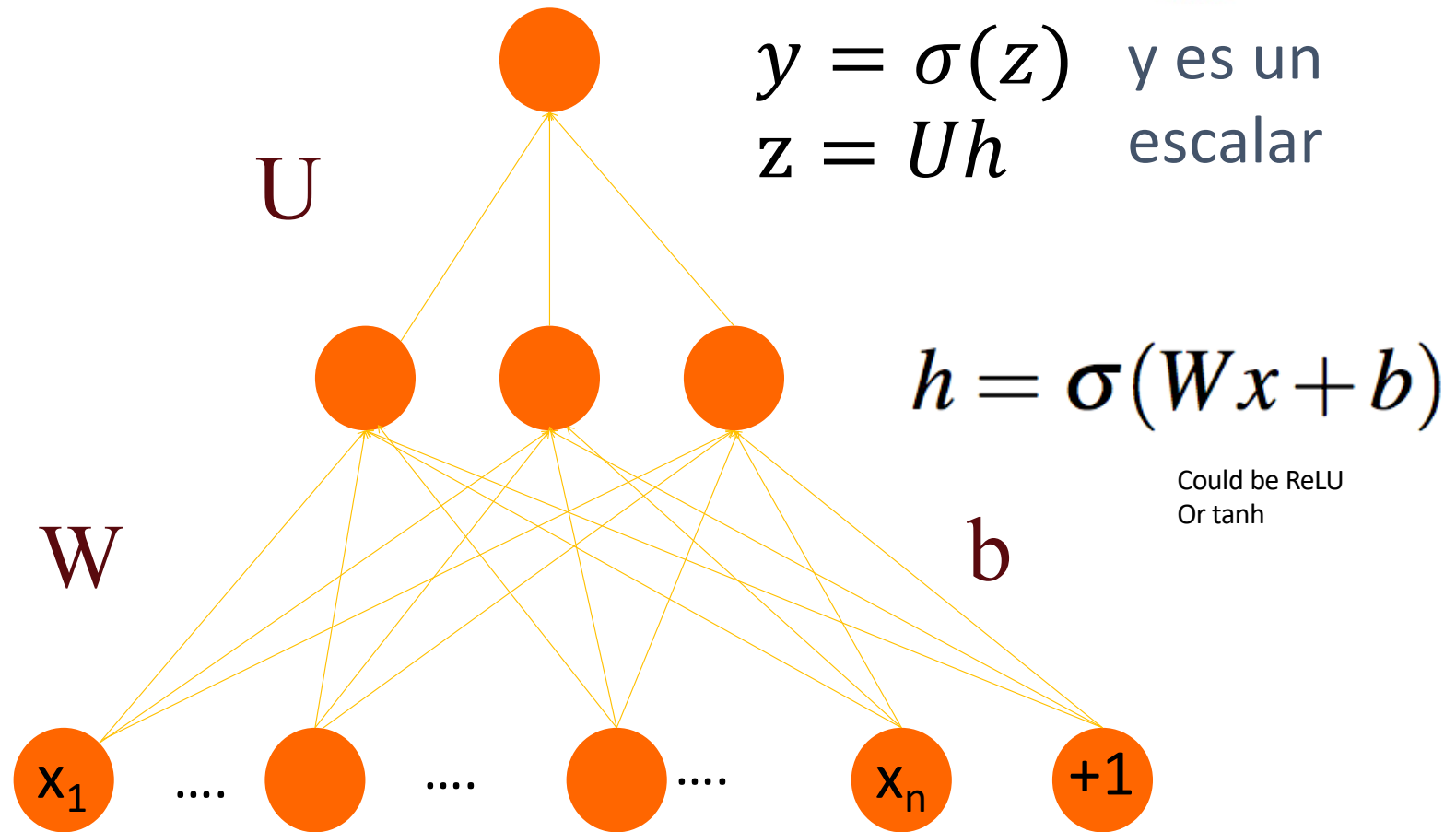


Máster en
Inteligencia
Artificial
UMU

Output layer
(σ node)

hidden units
(σ node)

Input layer
(vector)



Red de dos capas con salida escalar



Máster en
Inteligencia
Artificial
UMU

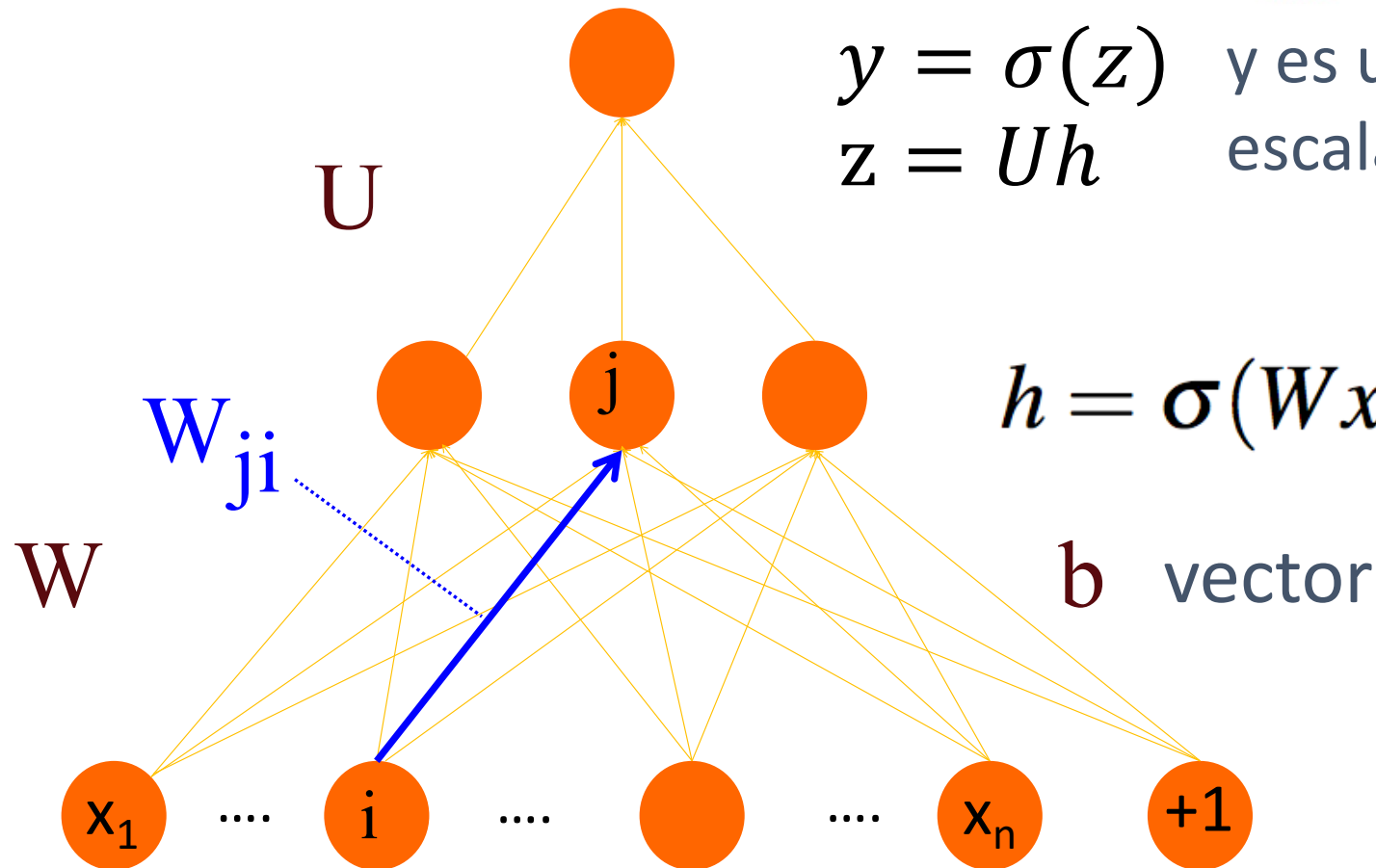
Output layer
(σ node)

$$y = \sigma(z) \quad y \text{ es un escalar}$$
$$z = Uh$$

hidden units
(σ node)

$$h = \sigma(Wx + b)$$

Input layer
(vector)



Red de dos capas con salida escalar



Máster en
Inteligencia
Artificial
UMU

Output layer
(σ node)

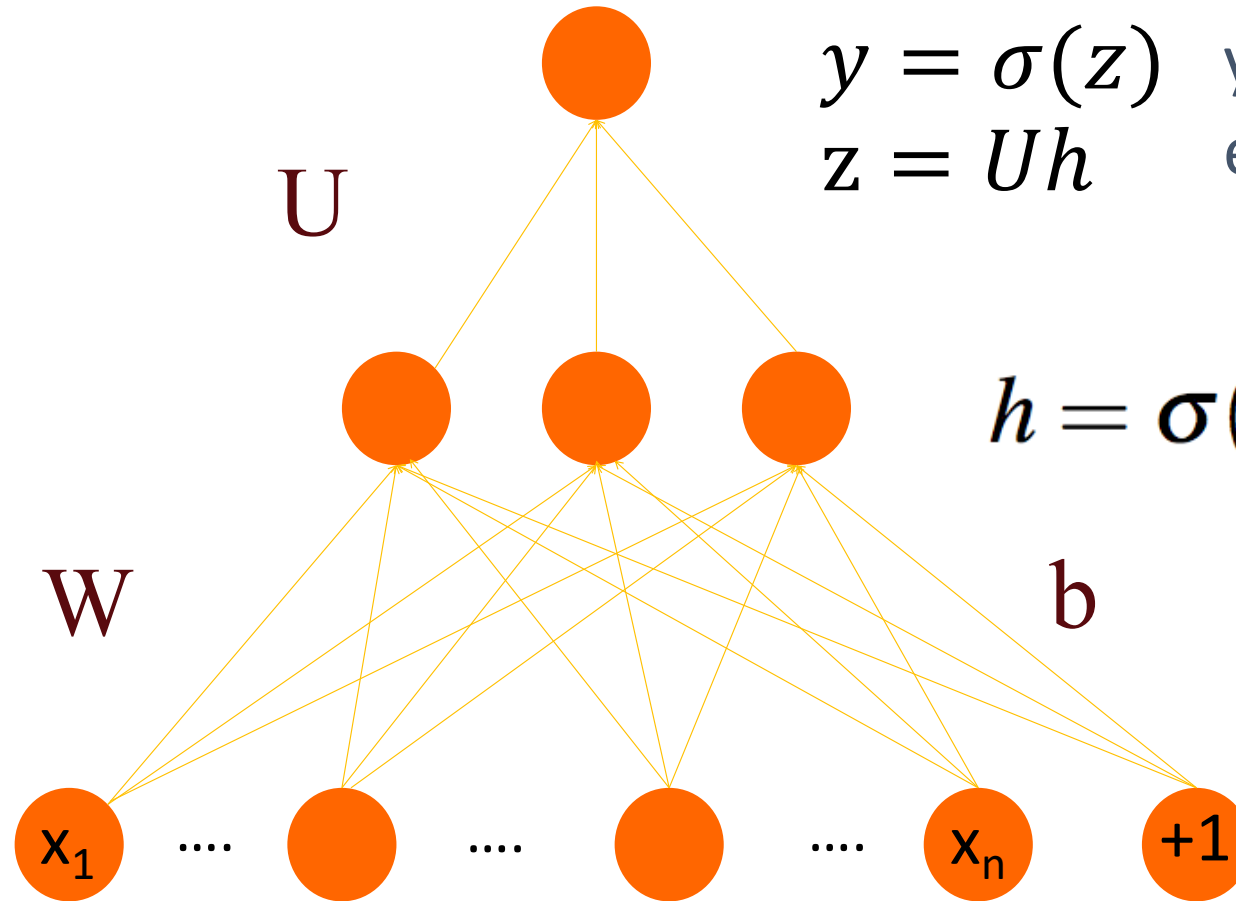
$$y = \sigma(z) \quad y \text{ es un escalar}$$
$$z = Uh$$

hidden units
(σ node)

$$h = \sigma(Wx + b)$$

Could be ReLU
Or tanh

Input layer
(vector)

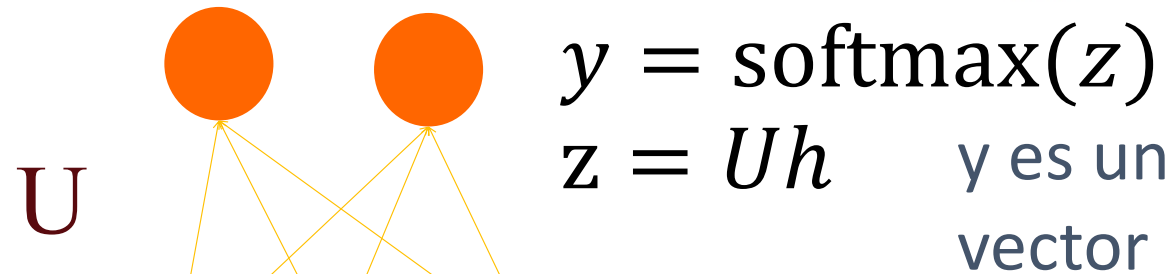


Red de dos capas con salida softmax

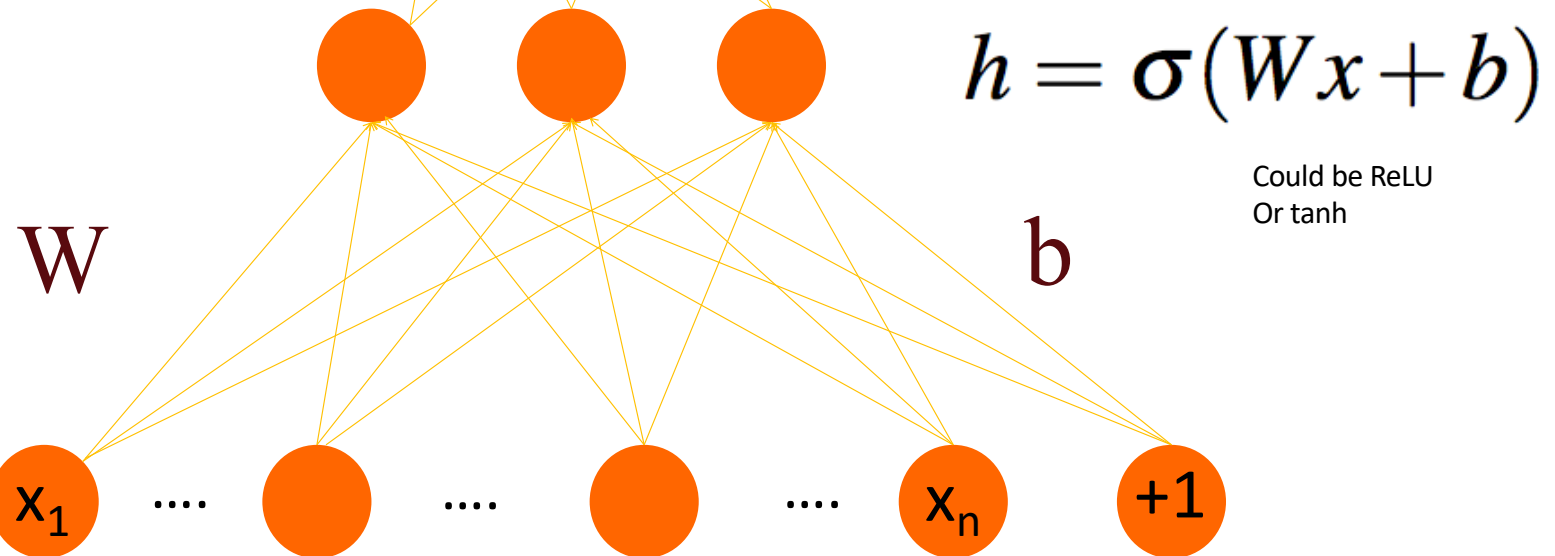


Máster en
Inteligencia
Artificial
UMU

Output layer
(σ node)



hidden units
(σ node)



Input layer
(vector)



Notación multi-capas (siempre la misma operación, independiente de la capa)



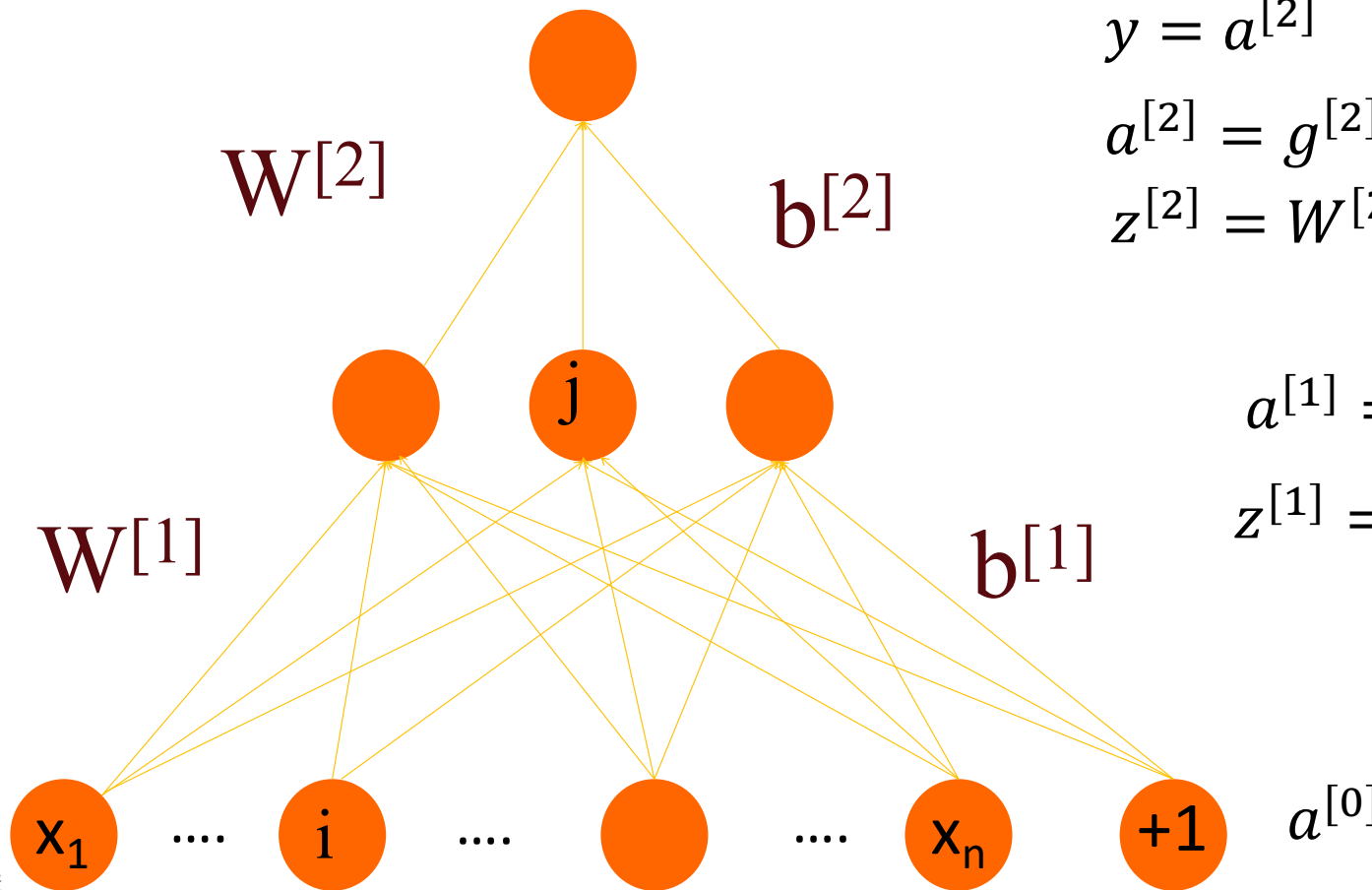
$$y = a^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]}) \text{ sigmoide / softmax}$$

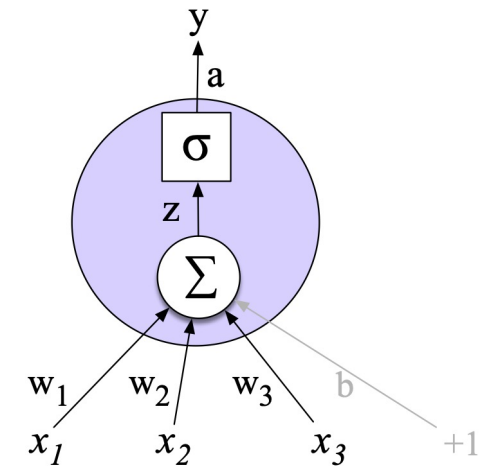
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[1]} = g^{[1]}(z^{[1]}) \text{ ReLU}$$

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$



Notación multi-capa: siempre la misma operación, independiente de la capa



$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$\hat{y} = a^{[2]}$$

for i in $1..n$

$$z^{[i]} = W^{[i]}a^{[i-1]} + b^{[i]}$$

$$a^{[i]} = g^{[i]}(z^{[i]})$$

$$\hat{y} = a^{[n]}$$



Disfrazar el sesgo para simplificar la notación

- Con una sencilla modificación, vamos a conseguir una notación mucho más sencilla, que trata a todos los pesos, incluido el sesgo, por igual
- Solo un cambio en la notación
 1. Añadimos un nodo dummy $a_0=1$ (sin efecto) a cada capa
 2. El sesgo de la capa será su peso, w_0
 3. Para la capa de entrada, $a^{[0]}_0=1$,
 - Además $a^{[1]}_0=1$, $a^{[2]}_0=1, \dots$



Disfrazar el sesgo para simplificar la notación

- En lugar de:

$$x = x_1, x_2, \dots, x_{n_0}$$

$$h = \sigma(Wx + b)$$

$$h_j = \sigma \left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j \right)$$

Hacemos:

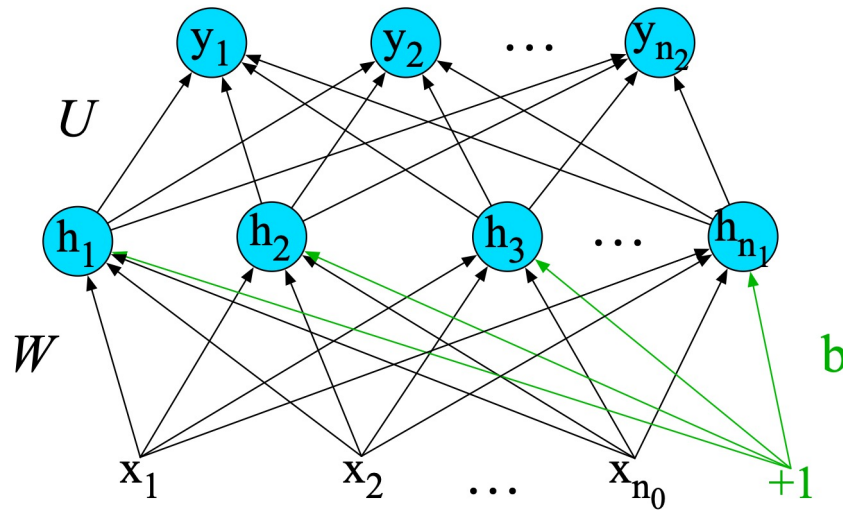
$$x = x_0, x_1, x_2, \dots, x_{n_0}$$

$$h = \sigma(Wx)$$

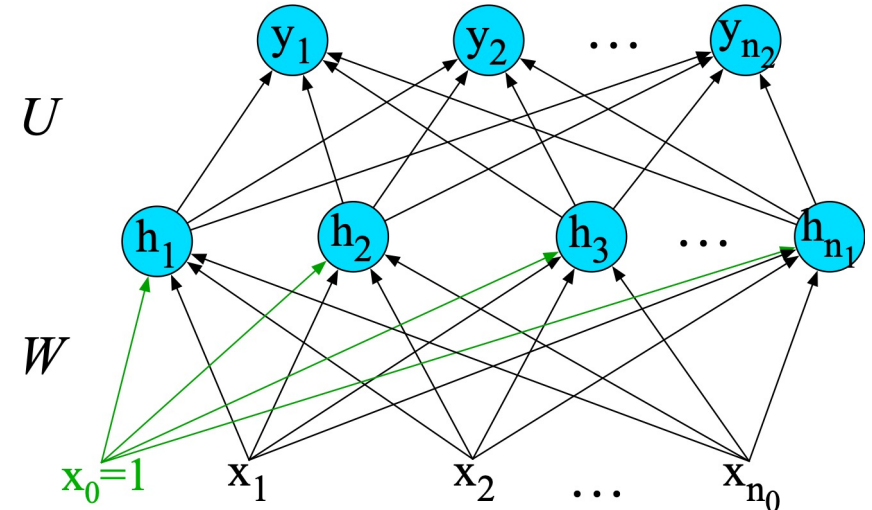
$$\sigma \left(\sum_{i=0}^{n_0} W_{ji} x_i \right)$$

Disfrazar el sesgo para simplificar la notación

En lugar de:



Hacemos:





Para entenderlas mejor

<https://playground.tensorflow.org>



Aplicando redes neuronales feed forward a tareas de procesamiento de lenguaje natural



Aplicando redes neuronales feed forward a tareas de procesamiento de lenguaje natural

Análisis de sentimientos

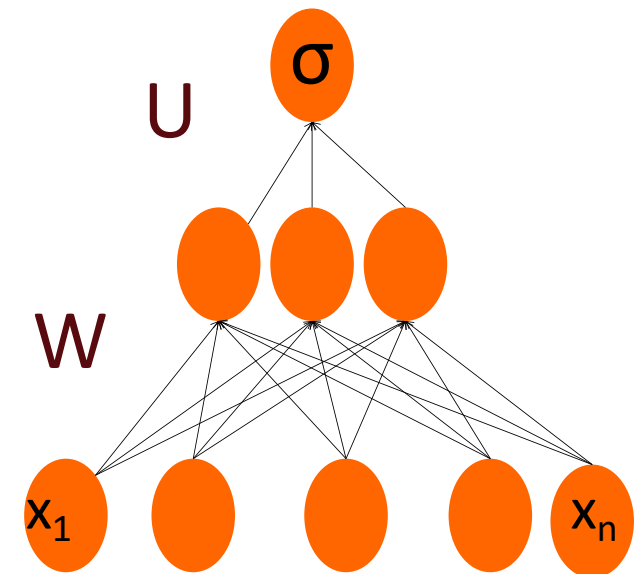


Tarea de clasificación: análisis de sentimientos

- Las características a la entrada de la red tendrían la siguiente definición

Var	Definition
x_1	$\text{count}(\text{positive lexicon}) \in \text{doc}$
x_2	$\text{count}(\text{negative lexicon}) \in \text{doc}$
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	$\text{count}(\text{1st and 2nd pronouns}) \in \text{doc}$
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	$\log(\text{word count of doc})$

- Doc se refiere al documento
- Positive lexicon es el léxico de referencia de palabras positivas
- Análogamente para el negative lexicon
- 1st and 2nd pronouns se refiere a primera y segunda persona



- La salida será 0 (review negativa) ó 1 (positiva)



Ejemplo

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

It's **hokey**. There are virtually **no** surprises, and the writing is **second-rate**.
So why was it so **enjoyable**? For one thing, the cast is
great. Another **nice** touch is the music. **I** was overcome with the urge to get off
the couch and start dancing. It sucked **me** in, and it'll do the same to **you**.

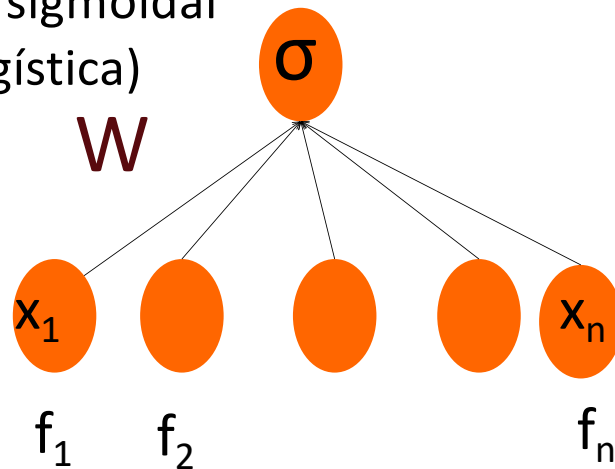
$x_2=2$
 $x_3=1$
 $x_1=3$
 $x_5=0$
 $x_6=4.19$
 $x_4=3$

Redes feedforward para clasificación simple

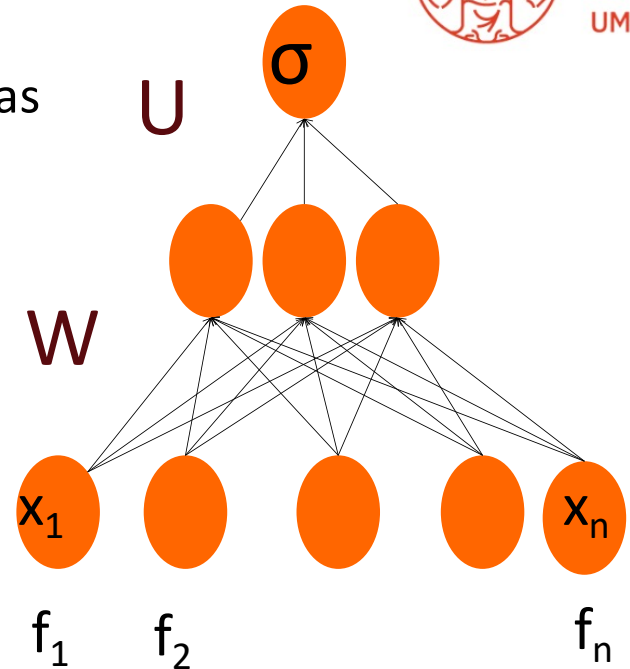


Máster en
Inteligencia
Artificial
UMU

Perceptrón + sigmoidal
(regresión logística)



MLP de dos capas



55

- Obtenemos el MLP añadiendo una CAPA OCULTA al perceptron + sigmoide
- Con esto modelamos interacciones no lineales entre los predictors de entrada
- Esto puede, o no, ser conveniente para una mejor resolución del problema

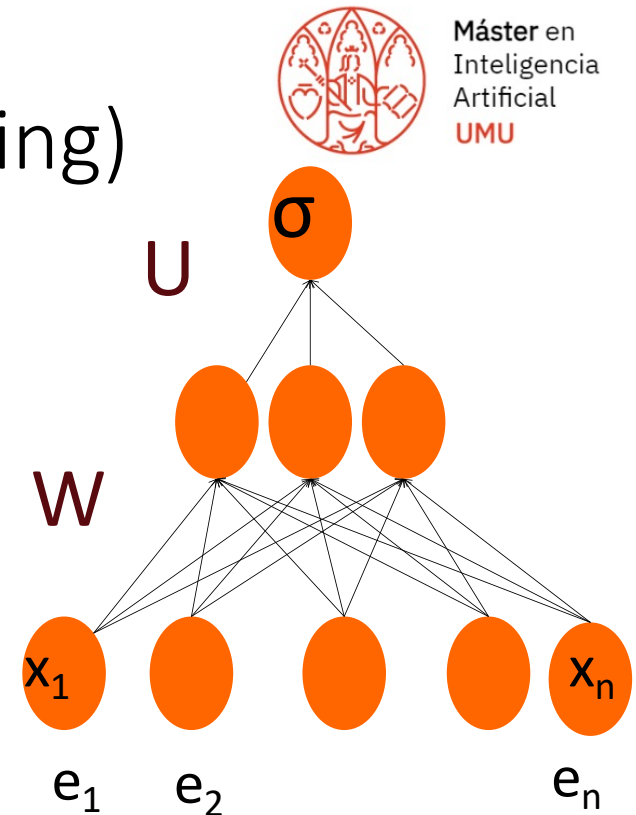


Aprendiendo los predictores automáticamente (representation learning)

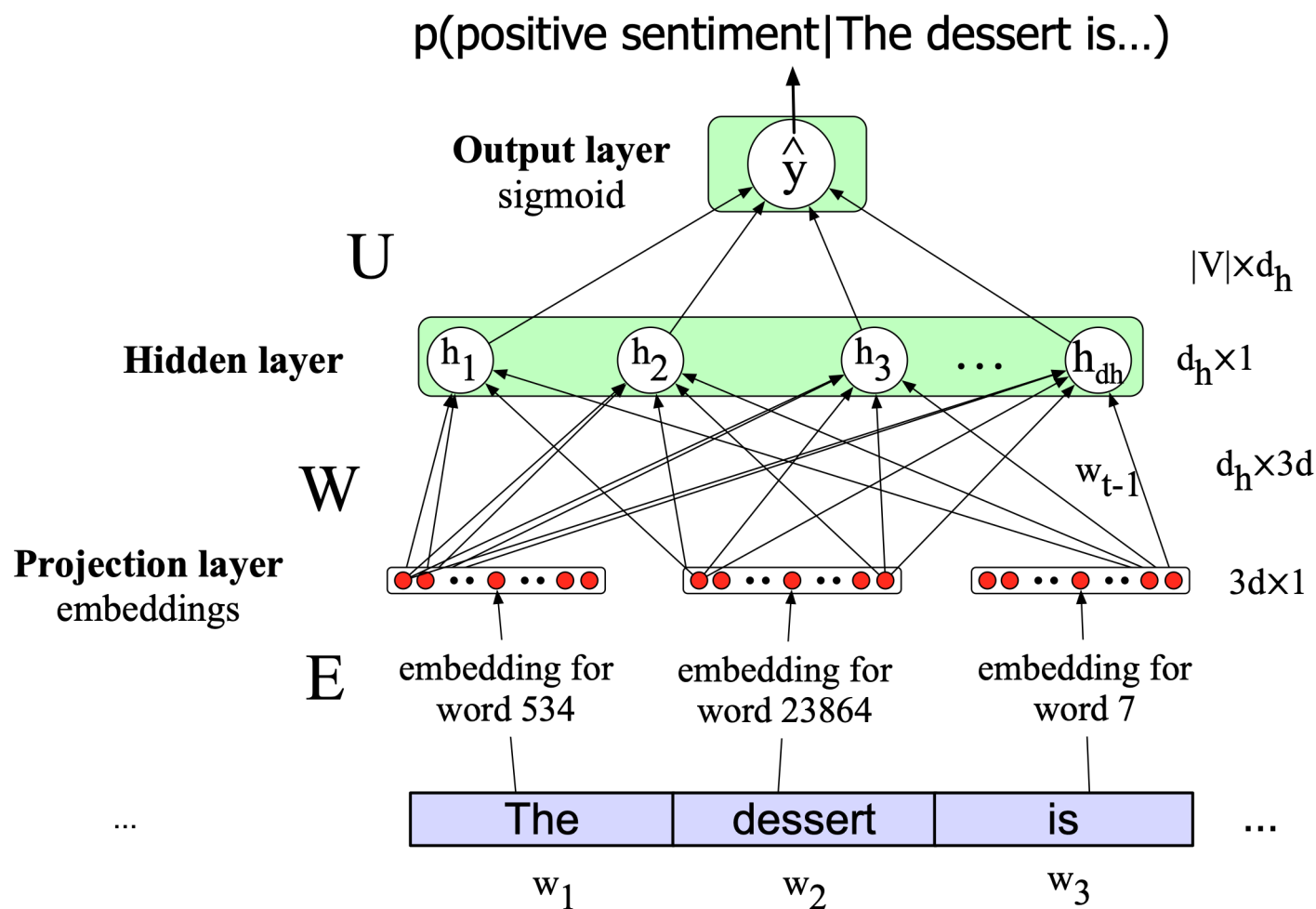
- En la solución anterior, los predictores ($x_1, \dots, x_6$) estaban diseñados a mano, no los hemos cuestionado

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

- Uno de los grandes avances del deep learning es el aprendizaje de representaciones, en donde la red neuronal aprende por sí misma la mejor representación (embedding) del espacio de predictores



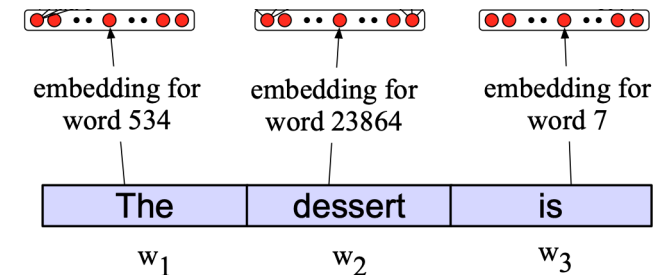
¿Cómo sería una red neuronal aplicadas a tareas NLP usando embeddings?





Pero... los documentos tienen diferentes tamaños!!!

- El ejemplo que hemos usado assume tamaño 3
- No es demasiado útil
- Algunas soluciones (lo mejor vendrá después)



1. Transformar la entrada a una longitud fija
 - Si la original es más corta, rellenar con ceros
 - Truncar si es más larga
2. En lugar de usar embeddings por palabras, crear un embedding por sentencia
 - Usar la media de los embeddings de todas sus palabras
 - Usar el máximo, componente a componente, de los embeddings de todas las palabras



Aplicando redes neuronales feed forward a tareas de procesamiento de lenguaje natural

Modelos de lenguaje



Modelos de lenguaje neuronales

- **Modelos de lenguaje:** estiman la probabilidad de la siguiente palabra en una secuencia

Los modelos de lenguaje neuronales actuales se basan en transformers, que ya veremos.

Los MLPs pueden ser casi tan buenos como los transformers, pero solo casi

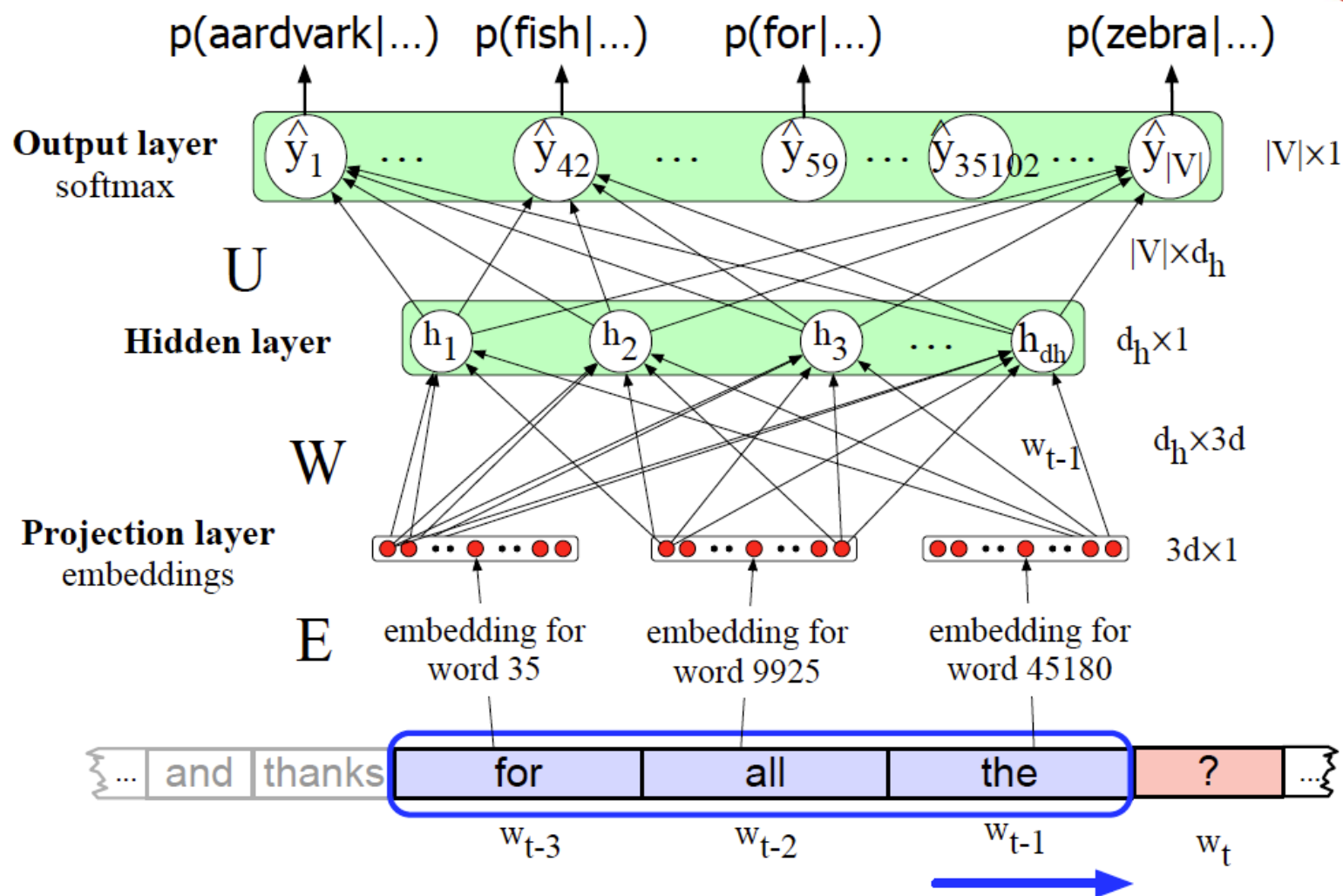


Modelos de lenguaje neuronales simples

- **Tarea:** predecir la siguiente palabra w_t dada la secuencia de palabras anteriores $w_{t-1}, w_{t-2}, w_{t-3}, \dots$
- **Problema:** en este caso, las secuencias tienen una longitud variable.
- **Solución:** usamos ventanas deslizantes de longitud fija (N)

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-N+1}^{t-1})$$

Modelo de lenguaje neuronal





Entrenando redes neuronales, una overview



¿De dónde salen los W's?

- En los problemas de clasificación:
- Conocemos la etiqueta correcta y (0, 1) para cada x .
- Pero nuestra red neuronal produce una estimación, $\hat{y}$
- Lo que buscamos es fijar los w y b tal que la **distancia** entre nuestra estimación $\hat{y}^{(i)}$ y el $y^{(i)}$ verdadero se minimice (tienda a 0)
- Necesitamos estimar la distancia mediante una función de pérdida (**loss function**)
- Necesitamos también un algoritmo para buscar los w , b que minimicen dicha función.



Nuestro sistema de machine learning necesita de dos componentes

- Una función de pérdida:
 - **Entropía cruzada (cross-entropy loss)**
- Un algoritmo de optimización:
 - **Gradiente descendente estocástico (stochastic gradient descent, SGD)**



Pérdida de una sola observación

- **Objetivo del proceso de entrenamiento:** maximizar la probabilidad $p(y|x)$
- Dado que solo hay dos posibles respuestas a generar por el clasificador, la probabilidad $p(y|x)$ del clasificador se deduce de la binomial, y viene dada por

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

- Si $y=1$, esa expresión es $\hat{y}$
- Si 0 , esa expresión es $1 - \hat{y}$
- Si aplicamos logaritmos a esa expresión, tenemos

$$\begin{aligned}\log p(y|x) &= \log [\hat{y}^y (1 - \hat{y})^{1-y}] \\ &= y \log \hat{y} + (1 - y) \log(1 - \hat{y})\end{aligned}$$



Función de pérdida total

Objetivo: maximizar la probabilidad de que el clasificador genere la etiqueta correcta, $p(y|x)$

Maximizar: $\log p(y|x) = \log [\hat{y}^y (1 - \hat{y})^{1-y}]$
 $= y \log \hat{y} + (1 - y) \log(1 - \hat{y})$

- Pero como estamos definiendo una pérdida, lo transformamos en algo que minimizar (cambiamos el signo)
- **Cross-entropy loss**

Minimizar: $L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$

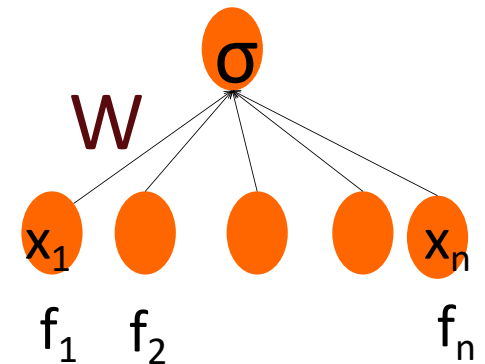
Ejemplo, análisis de sentimientos y un perceptron con salida sigmoidea



Máster en
Inteligencia
Artificial
UMU

It's hokey . There are virtually no surprises , and the writing is second-rate .
So why was it so enjoyable ? For one thing , the cast is
great . Another nice touch is the music . I was overcome with the urge to get off
the couch and start dancing . It sucked me in , and it'll do the same to you .

¿Es una review positiva (0) o negativa (1)?





Recordemos los predictores

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)



Ejemplo, análisis de sentimientos y un perceptron con salida sigmoidea

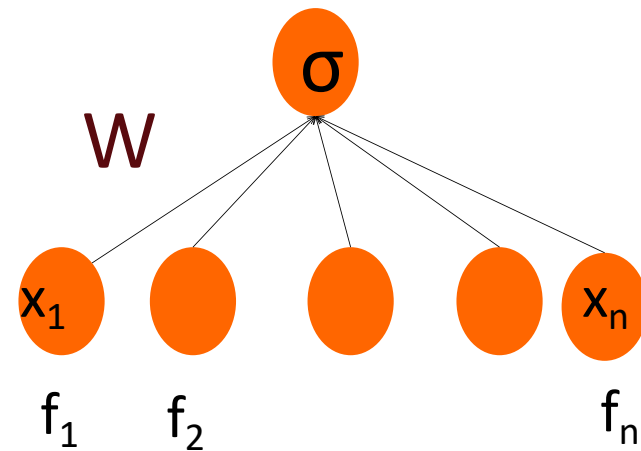
It's **hokey**. There are virtually **no** surprises, and the writing is **second-rate**.
So why was it so **enjoyable**? For one thing, the cast is
great. Another **nice** touch is the music. **I** was overcome with the urge to get off
the couch and start dancing. It sucked **me** in, and it'll do the same to **you**.

$x_1=3$ $x_2=2$ $x_3=1$ $x_4=3$ $x_5=0$ $x_6=4.19$



Nuestra neurona sigmoideal totalmente definida

- Supongamos que los pesos W , b de nuestro perceptrón + sigmoide son $w=[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$, $b=0.1$



Vamos a verlo dependiendo del valor verdadero de la etiqueta del documento



- Si el valor verdadero es $y=1$. ¿Cómo de bien lo hace nuestro modelo?

$$\begin{aligned} p(+|x) &= P(Y = 1|x) & \hat{y} &= \sigma(w \cdot x + b) \\ & & &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\ & & &= \sigma(.833) \\ & & &= 0.70 \end{aligned} \tag{5.6}$$

- Bastante bien, ¿qué pérdida genera eso?

$$\begin{aligned} L_{CE}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log \sigma(w \cdot x + b)] \\ &= -\log(.70) \\ &= .36 \end{aligned}$$

Vamos a verlo dependiendo del valor verdadero de la etiqueta del document (y 2)



- Supongamos que el valor verdadero fuera $y=0$.

$$p(-|x) = P(Y = 0|x) \quad \hat{y} = 1 - \sigma(w \cdot x + b) \\ = 0.30$$

- ¿Qué pérdida generaría?

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log (1 - \sigma(w \cdot x + b))] \\ &= -\log (.30) \\ &= 1.2 \end{aligned}$$

- Suponiendo que la review fuera positiva, nuestro modelo se comporta bastante bien, al menos para este documento. Pues genera una pérdida menor (0.36) cuando acierta que cuando se equivoca (1.2)



Gradiente descendente estocástico (SGD), buscando los pesos adecuados



Objetivo: minimizar la pérdida

- Como ahora los parámetros de nuestro modelo (los pesos) son muy importantes pues son los que tenemos que variar para optimizar la función de pérdida, necesitamos expresarlos de alguna forma
- Los denotamos con $\theta=(w,b)$
- Ahora, la salida de nuestro modelo, $\hat{y}$, la representaremos con $f(x; \theta)$ así los parámetros son visibles en la notación
- Ahora, nuestro problema se reduce a esta expression, que se traduce en hallar el vector (o matriz) de valores θ que minimiza L_{CE} promediado en los m ejemplos de entrenamiento

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{CE}(f(x^{(i)}; \theta), y^{(i)})$$

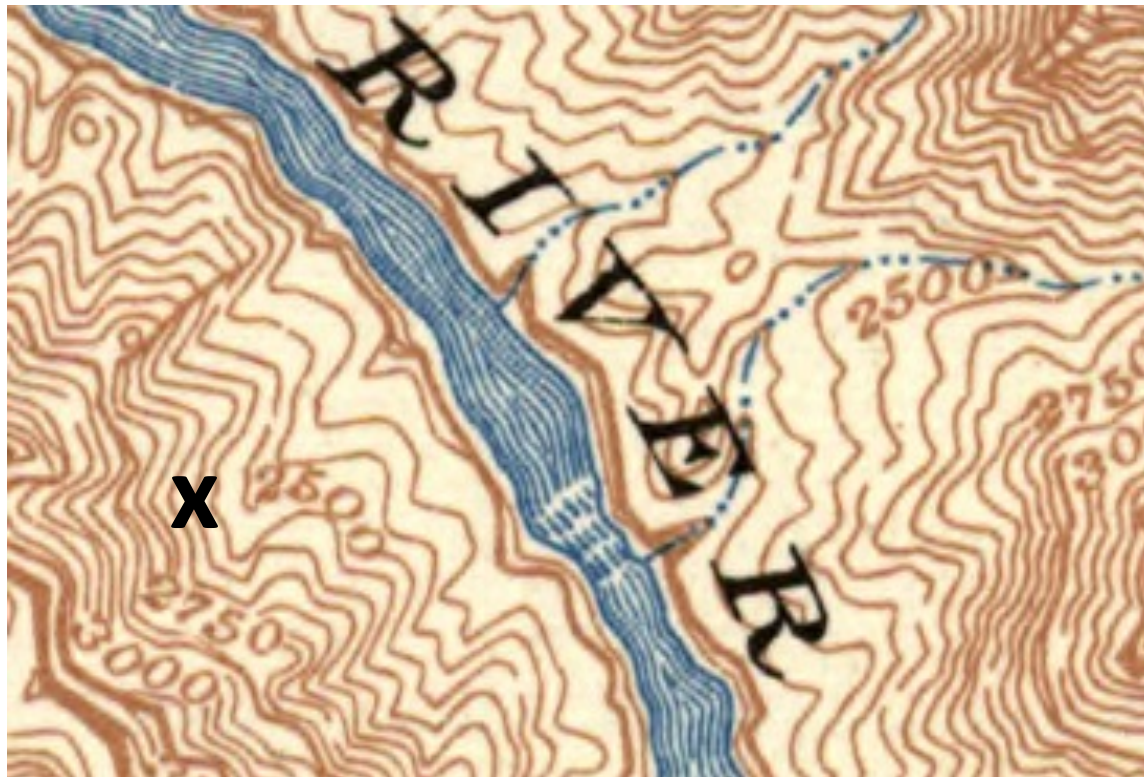


SGD (Stochastic Gradient Descent): el algoritmo más conocido del deep learning



Explicación intuitiva del SGD

- ¿Cómo llego al río desde cualquier punto del cañón (X) si no dispongo del mapa?



Heurística: miro a mi alrededor 360°, encuentro la dirección de bajada más pronunciada y sigo por ese camino.



Si queremos usar SGD, tenemos que transformar nuestro problema en un problema basado en curvas de nivel



Vuelta a la función de pérdida

- La función de pérdida definirá nuestras curvas de nivel
 - Mayor profundidad, más cercanos al río, menos pérdida
- Idealmente, la función de pérdida debería ser convexa
 - un único mínimo
 - Empecemos donde empecemos nuestro “descenso al río”, siempre llegaremos al mínimo
- La función de pérdida depende del modelo que usemos, recordemos que

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$$

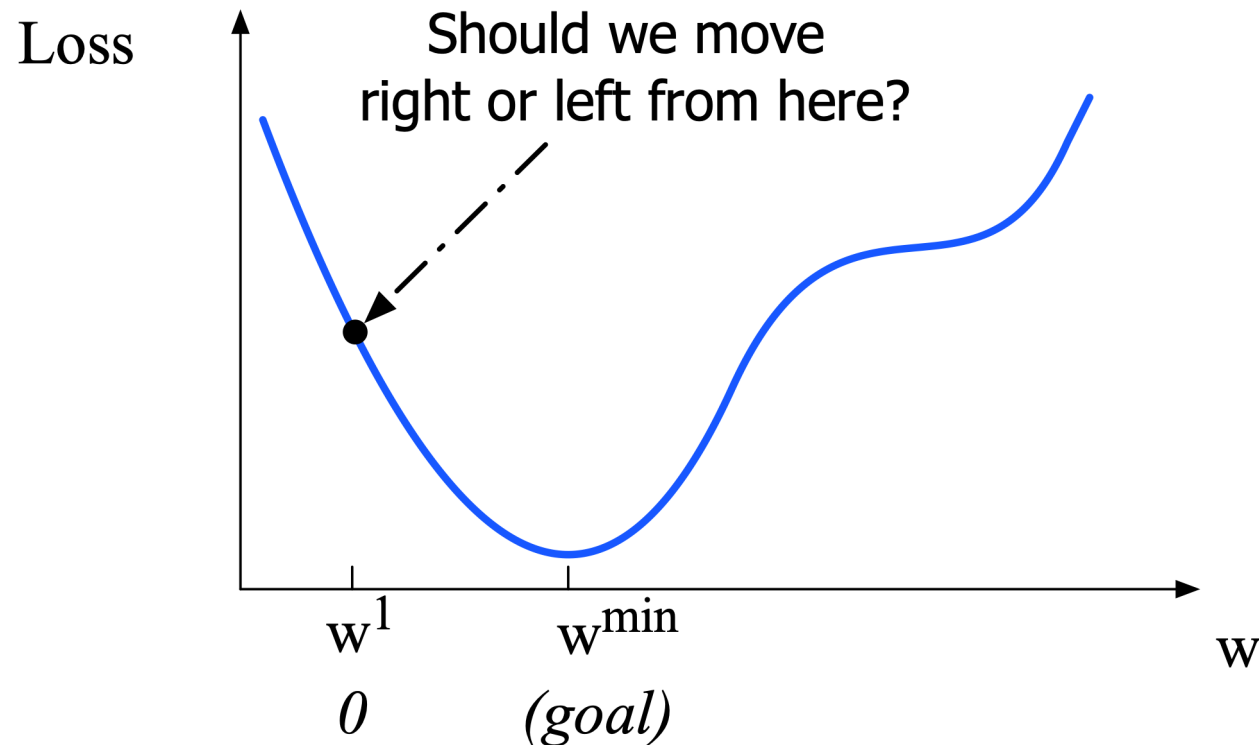
Vamos a visualizar el problema para un solo peso w



Máster en
Inteligencia
Artificial
UMU

Q: Dado el valor de w actual, ¿lo aumentamos o lo disminuimos?

A: Deberíamos actualizar w en el sentido inverso a la pendiente de la función



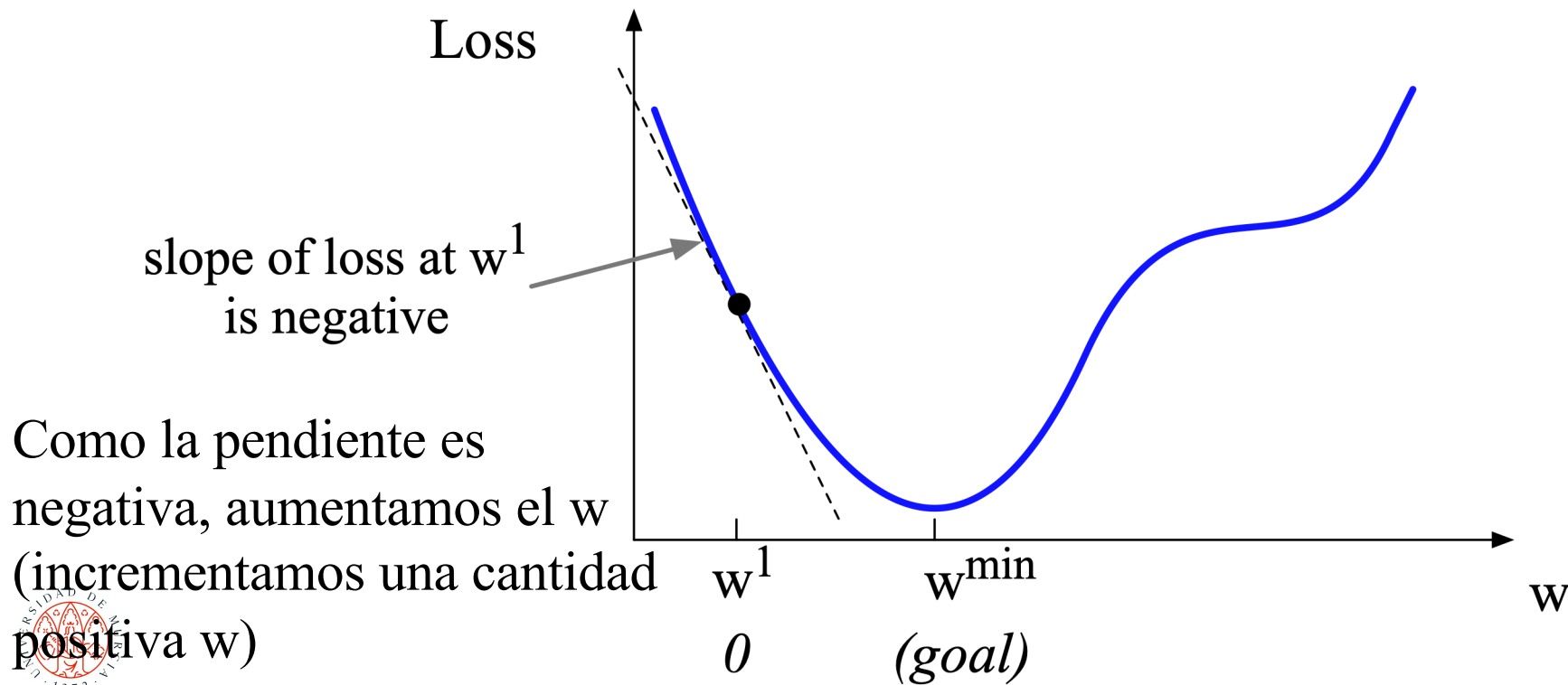
Vamos a visualizar el problema para un solo peso w



Máster en
Inteligencia
Artificial
UMU

Q: Dado el valor de w actual, ¿lo aumentamos o lo disminuimos?

A: Deberíamos actualizar w en el sentido inverso a la pendiente de la función



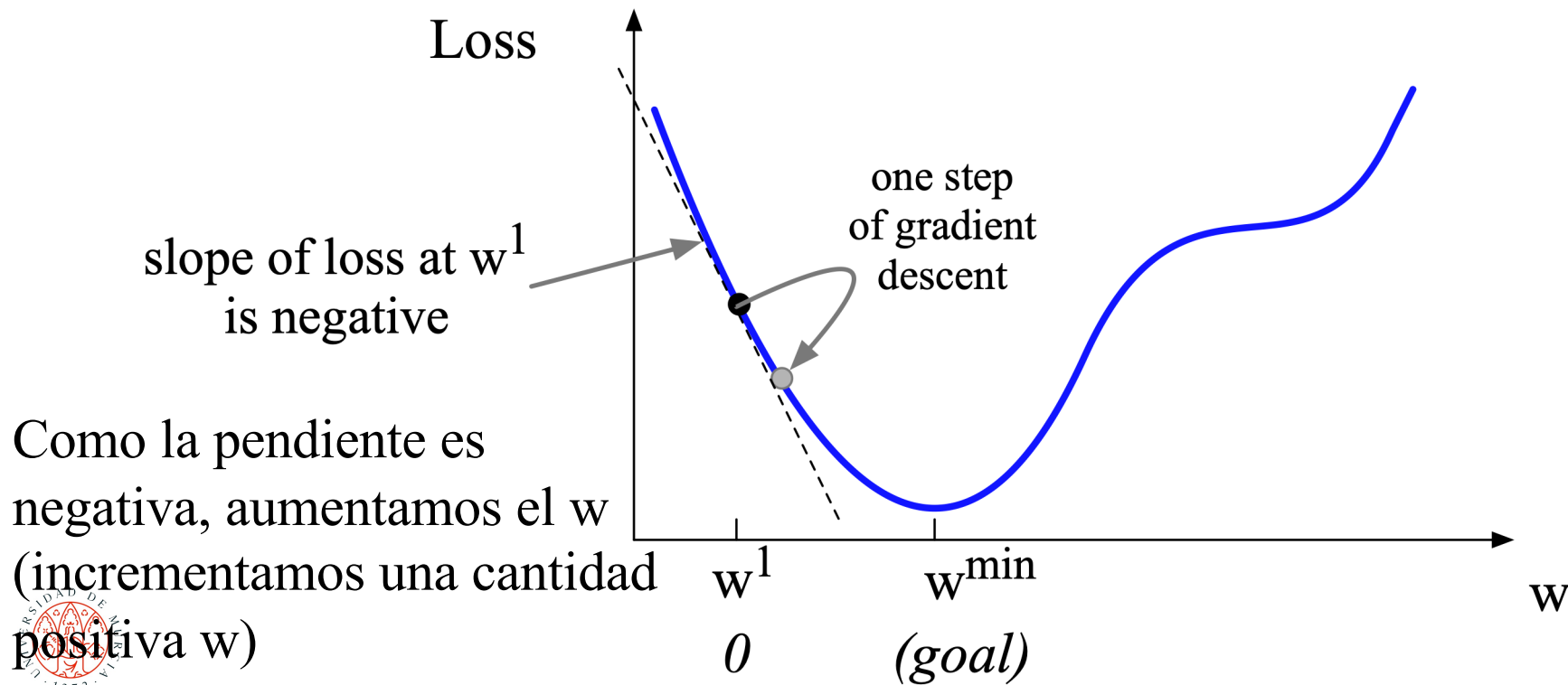
Vamos a visualizar el problema para un solo peso w



Máster en
Inteligencia
Artificial
UMU

Q: Dado el valor de w actual, ¿lo aumentamos o lo disminuimos?

A: Deberíamos actualizar w en el sentido inverso a la pendiente de la función





Está claro: la clave es calcular la pendiente de la función de pérdida!

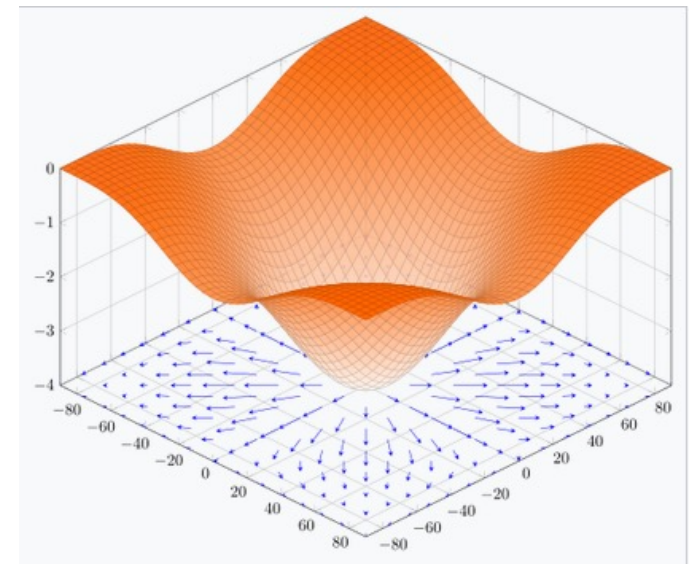


Para eso usamos el operador de gradiente

El **gradiente** de una función de n variables es un vector que apunta en la dirección del mayor incremento del valor de la función,

Viene dado por la derivada parcial de la pérdida con respecto a esas n variables (los pesos)

$$\frac{d}{dw} L(f(x; w), y)$$



The gradient of the function $f(x,y) = -(\cos^2 x + \cos^2 y)^2$ depicted as a projected **vector field** on the bottom plane.

<https://en.wikipedia.org/wiki/Gradient>



Para eso usamos el operador de gradiente

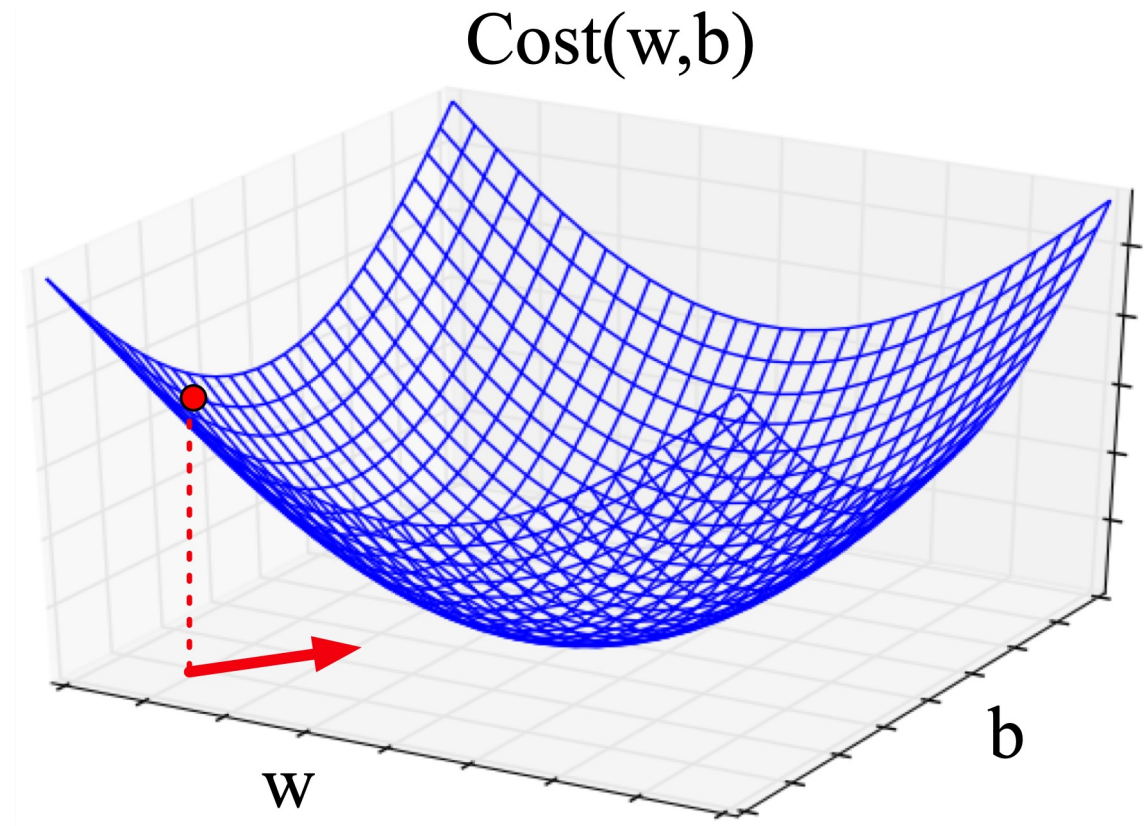
- El algoritmo SGD
 - Usa el gradiente de la función de pérdida en el instante actual
 - Viene dado por los pesos de nuestra red neuronal
 - Los aplica a los propios pesos para moverlos en el espacio
- Hay que determinar qué cantidad de movimiento aplicar

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

Si damos el salto a dos dimensiones,
la visualización sería

- El vector de pesos de la red nos sitúa en el punto rojo
- La función de pérdida es la superficie (convexa) azul
- Solo tiene un peso y un sesgo
- SGD variaría los dos hasta intentar llegar al fondo de la convexidad

2025/2026





En los modelos de redes deep, los gradientes...

- Son mucho más grandes, de miles, cientos de miles, millones, cientos de millones de parámetros en los LLMs (Large Language Models)
- En cada dimensión l , para w_i el componente correspondiente del gradiente i indica la pendiente de la función de pérdida con respecto a esa variable (en qué medida un cambio pequeño en w_i influencia la función de pérdida global L)
- Dicha pendiente a nivel de componente es una derivada parcial ∂ , con respect al peso, ∂w_i
- Por tanto, el gradiente final se define como un vector de derivadas parciales



Entonces, la función de pérdida es n -dimensional, siendo n muy grande



El gradiente es entonces el Jacobiano

Retomamos la expression de $\hat{y}$ como $f(x; \theta)$:

$$\nabla_{\theta} L(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x; \theta), y) \\ \frac{\partial}{\partial w_2} L(f(x; \theta), y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x; \theta), y) \end{bmatrix}$$

Así, la expression final de actualización de pesos θ es simple

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

El algoritmos SGD



function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x , y) **returns** θ

where: L is the loss function

f is a function parameterized by θ

x is the set of training inputs $x^{(1)}, x^{(2)}, \dots, x^{(m)}$

y is the set of training outputs (labels) $y^{(1)}, y^{(2)}, \dots, y^{(m)}$

$\theta \leftarrow 0$

repeat til done # see caption

For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)

1. Optional (for reporting): # How are we doing on this tuple?

 Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ # What is our estimated output $\hat{y}$?

 Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$ # How far off is $\hat{y}^{(i)}$ from the true output $y^{(i)}$?

2. $g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$ # How should we move θ to maximize loss?

3. $\theta \leftarrow \theta - \eta g$ # Go the other way instead

return θ



SGD: un ejemplo



Ejemplo de aplicación de GD

- Vamos a ver cómo se aplica un paso
- Usaremos un ejemplo, de análisis de sentimientos, con un ejemplo positive ($y=1$)
- Dos predictores:

$x_1 = 3$ (número de palabras positivas en nuestro lexicón)

$x_2 = 2$ (número de palabras negativas en nuestro lexicón)

Asumimos entonces 3 parametros (2 w 's, 1 b) todos 0 en Θ^0 :

$$w_1 = w_2 = b = 0$$

$$\eta = 0.1$$



Ejemplo de aplicación de GD

- El paso de actualización de pesos es:

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

$$w_1 = w_2 = b = 0;$$
$$x_1 = 3; \quad x_2 = 2$$

- Siendo

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

- Y el vector gradiente tiene 3 dimensiones:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix}$$



Ejemplo de aplicación de GD

- El paso de actualización de pesos es:

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

$$w_1 = w_2 = b = 0;$$
$$x_1 = 3; \quad x_2 = 2$$

- Siendo

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

- Y el vector gradiente tiene 3 dimensiones:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} \\ \\ \end{bmatrix}$$



Ejemplo de aplicación de GD

- El paso de actualización de pesos es:

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

$$w_1 = w_2 = b = 0;$$
$$x_1 = 3; \quad x_2 = 2$$

- Siendo

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

- Y el vector gradiente tiene 3 dimensiones:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} :$$



Ejemplo de aplicación de GD

- El paso de actualización de pesos es:

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

$$w_1 = w_2 = b = 0;$$
$$x_1 = 3; \quad x_2 = 2$$

- Siendo

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

- Y el vector gradiente tiene 3 dimensiones:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} =$$



Ejemplo de aplicación de GD

- El paso de actualización de pesos es:

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

$$w_1 = w_2 = b = 0;$$
$$x_1 = 3; \quad x_2 = 2$$

- Siendo

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

- Y el vector gradiente tiene 3 dimensiones:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$



Ejemplo de aplicación de GD

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Una vez calculado el vector gradiente, obtenemos el nuevo vector de parámetros θ^1 moviendo θ^0 en la dirección opuesta del gradiente, pero solo la fracción que nos permite el learning rate (10%):

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y) \quad \eta = 0.1;$$

$$\theta^1 =$$



Ejemplo de aplicación de GD

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Una vez calculado el vector gradiente, obtenemos el nuevo vector de parámetros θ^1 moviendo θ^0 en la dirección opuesta del gradiente, pero solo la fracción que nos permite el learning rate (10%):

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y) \quad \eta = 0.1;$$

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$



Ejemplo de aplicación de GD

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Una vez calculado el vector gradiente, obtenemos el nuevo vector de parámetros θ^1 moviendo θ^0 en la dirección opuesta del gradiente, pero solo la fracción que nos permite el learning rate (10%):

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y) \quad \eta = 0.1;$$

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$



Ejemplo de aplicación de GD

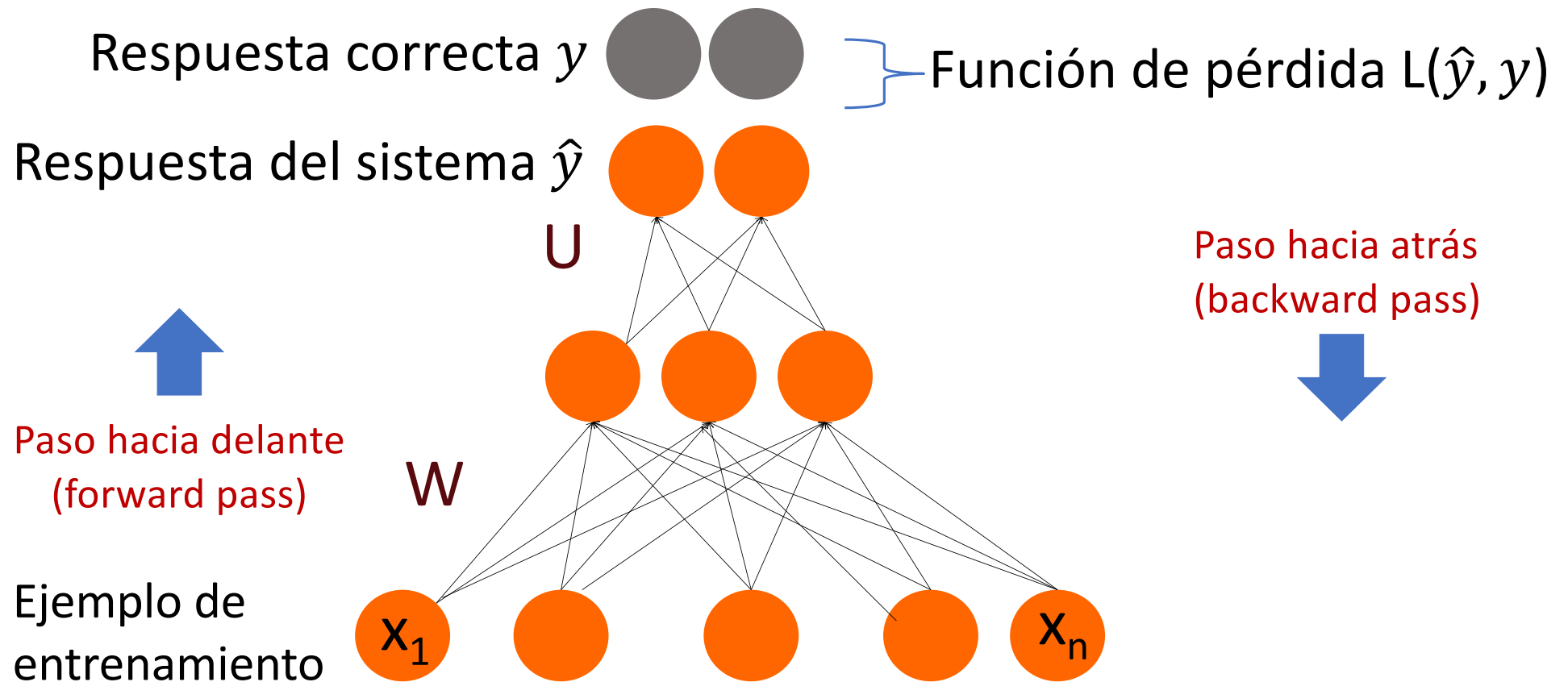
$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Una vez calculado el vector gradiente, obtenemos el nuevo vector de parámetros θ^1 moviendo θ^0 en la dirección opuesta del gradiente, pero solo la fracción que nos permite el learning rate (10%):

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y) \quad \eta = 0.1;$$

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$

Entrenando una red de dos capas





Entrenando una red de dos capas

- Para cada ejemplo de entrenamiento (x, y)
 - Ejecuta paso hacia delante (*forward* computation) para obtener $\hat{y}$
 - Ejecuta paso hacia atrás (*backward* computation) para actualizar los pesos w :
 - Para cada nodo de salida
 - Calcular pérdida L entre y e $\hat{y}$
 - Para cada peso w desde la capa oculta a la de salida
 - Actualizar peso su parte de la pérdida
 - Para cada nodo oculto
 - Calcular su contribución a la pérdida entre y e $\hat{y}$
 - Para cada w desde la capa de entrada a la oculta
 - Actualizar peso según su parte de la contribución



Conclusiones

- Los perceptrones tienen una capacidad de representación limitada
- Aplilar perceptrones añadiendo funciones de activación a cada capa es útil para nuestros propósitos
- Podemos entrenar redes neuronales con esas unidades y funciones de activación si las conectamos fully connected y feed forward



Referencias

Para preparar el material de esta asignatura se ha usado el libro de la asignatura de Jurafsky & Martin (Speech and Language Processing), capítulos 4 y 7

<https://web.stanford.edu/~jurafsky/slp3/>

Speech and Language Processing (3rd ed. draft)

Dan Jurafsky and **James H. Martin**

Here's our August 20, 2024 release!